

zkFL: Verifiable Byzantine-Robust Federated Learning Against Malicious Servers

Xiangyun Tang[✉], Member, IEEE, Minyang Li[✉], Tao Zhang[✉], Yijing Lin[✉], Member, IEEE, Liehuang Zhu[✉], Senior Member, IEEE, Chuan Zhou[✉], and Xiangzhi Liu

Abstract—In low-altitude networks, various aerial platforms such as autonomous aerial vehicles and airships cooperate to provide services including real-time monitoring, emergency response, and data collection. These platforms often operate with limited computing resources, restricted energy supply, and unstable wireless connectivity, making centralized data processing inefficient and prone to privacy risks. Federated Learning (FL) provides a promising solution by enabling multiple platforms to collaboratively train a shared model without exchanging raw data. However, the presence of Byzantine clients and a potentially malicious server poses serious threats to the robustness and trustworthiness of FL in such environments. Existing Byzantine-robust FL methods typically assume a semi-honest server and rely on auxiliary information such as clean datasets or known attacker ratios, which limits their applicability in dynamic and non-IID scenarios. In this paper, we propose zkFL, a Byzantine-robust FL framework that embeds zero-knowledge proofs to ensure verifiable aggregation under a malicious server. ZkFL allows clients to verify the correctness

of server-side aggregation and dynamically adjusts client weights based on inference-guided detection, without relying on external datasets. Each round includes a zk-SNARK proof to guarantee aggregation integrity while preserving gradient privacy. Experiments demonstrate that zkFL exhibits strong robustness and verifiability in both IID and non-IID settings, outperforming prior methods, even in the presence of a malicious server.

Index Terms—Federated learning, privacy preservation, zero-knowledge proof.

I. INTRODUCTION

LOW-ALTITUDE network recently plays an important role in supporting intelligent services such as disaster response, aerial surveillance, and emergency communication [1]. It relies on mobile platforms such as autonomous aerial vehicles (AAVs), balloons, and lightweight airships to establish flexible and rapidly deployable communication infrastructures between airborne nodes and ground users [1], [2], [3]. These platforms typically operate at relatively low altitudes and form a dynamic and distributed communication environment. Compared with traditional ground-based networks, low-altitude aerial platforms provide broader coverage and higher mobility due to their flexible deployment and elevated perspective [4], [5], [6]. However, in distributed environments, these platforms exhibit unstable connectivity, limited computing and energy resources, and frequent changes in network topology [7], [8].

Recently, Federated Learning (FL) has emerged as a distributed machine learning paradigm to allow multiple aerial platforms (i.e., clients) in low-altitude networks to collaboratively train models while keeping their private data local, as shown in Fig. 1 [9], [10]. In low-altitude networks with distributed and mobile platforms, each client performs local training and periodically transmits model updates to a central coordinator (i.e., the server) to contribute to a shared global model. This training process avoids raw data exchange, reducing communication burden and enhancing data privacy [11], [12], [13]. However, in low-altitude networks, the FL process remains vulnerable to adversarial behaviors [14], [15], [16], [17], [18], [19]. Due to unstable connectivity, potential physical compromise of aerial devices, or software tampering during deployment and communication, certain clients may attempt to disrupt the training process by injecting false model updates (i.e., Byzantine clients). Such Byzantine attacks degrade global model accuracy, slow convergence, and introduce security risks that challenge the reliability of FL [20], [21], [22], [23].

Received 15 August 2025; revised 22 September 2025; accepted 13 October 2025. Date of publication 15 October 2025; date of current version 24 December 2025. This work was supported in part by the National Key R&D Program of China under Grant 2024YFB4709000, in part by the National Natural Science Foundation of China under Grant 62572502, Grant 62302539, and Grant 62402029, in part by the Hainan Provincial Natural Science Foundation Lingshui Li'an International Education Innovation Experimental Zone Joint Project under Grant 624LALH005, in part by the Beijing Natural Science Foundation under Grant L251041 and Grant L251038, in part by the Fujian Provincial Social Science Fund Youth Project under Grant FJ2024C017, in part by the fellowship of China National Postdoctoral Program for Innovative Talents under Grant BX20240045, in part by China Postdoctoral Science Foundation General Program under Grant 2025M773481, in part by the Key Laboratory of Computing Power Network and Information Security, Ministry of Education under Grant 2024ZD014, and in part by Taishan Industrial Experts Program under Grant tscy20231203. Recommended for acceptance by Dr. Yao Sun. (Corresponding authors: Chuan Zhou; Tao Zhang.)

Xiangyun Tang, Minyang Li, and Chuan Zhou are with the School of Information Engineering, Minzu University of China, and Key Laboratory of Ethnic Language Intelligent Analysis and Security Governance of MOE, Minzu University of China, Beijing 100081, China (e-mail: xiangyunt@muc.edu.cn; minyangl@muc.edu.cn; zhouchuan@tju.edu.cn).

Tao Zhang is with the School of Cyberspace Science and Technology, Beijing Jiaotong University, Beijing 100044, China (e-mail: taozh@bjtu.edu.cn).

Yijing Lin is with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: yjlin@bupt.edu.cn).

Liehuang Zhu is with the School of Cyberspace Security, Beijing Institute of Technology, Beijing 100044, China (e-mail: liehuangz@bit.edu.cn).

Xiangzhi Liu is with the Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), Shangdong 250316, China, also with the Shandong Provincial Key Laboratory of Industrial Network and Information System Security, Shandong Fundamental Research Center for Computer Science, Jinan 250002, China, and also with Shandong Shanke Intelligent Technology Company, Ltd., Jinan 250199, China (e-mail: Liuxzh@sdsas.org).

Digital Object Identifier 10.1109/TNSE.2025.3622060

TABLE I
COMPARISON OF BYZANTINE-ROBUST FEDERATED LEARNING METHODS

Method	Malicious Server	No Info Leakage	Low Computation	Robust in Non-IID	Auxiliary Information
<i>Statistical Aggregation Methods</i>					
Krum [10]	✗	✓	✓	✗	Requires attacker ratio
Multi-Krum [10]	✗	✓	✓	✗	Requires attacker ratio
Trimmed Mean [28]	✗	✓	✓	✗	Requires attacker ratio
Median [28]	✗	✓	✓	✗	Majority benign assumption
Bulyan [29]	✗	✓	✗	✗	Needs prior information
<i>Filter-based Methods</i>					
FLTrust [30]	✗	✓	✓	✗	Needs trusted dataset
Zeno++ [33]	✗	✓	✗	✗	Need validation loss
FoolsGold [34]	✗	✓	✓	✗	No
CareFL [31]	✗	✓	✓	✓	Small clean dataset
FedInv [32]	✗	✗	✓	✗	No
FLDetector [35]	✗	✗	✗	✓	Needs anomaly detector
FLGuard [36]	✗	✓	✗	✓	No
FedSmart [37]	✗	✓	✓	✓	Needs validation dataset
MUDGUARD [38]	✓	✓	✗	✓	No
FLAME [39]	✗	✓	✓	✓	No
EIFFeL [40]	✗	✓	✗	✓	No
<i>Asynchronous or Special Architectures</i>					
AFLGuard [41]	✗	✓	✗	✓	Needs trusted dataset
BRACE [42]	✗	✓	✓	✓	No
DFL-Dual [43]	✗	✓	✗	✓	No
FedCmp [44]	✗	✓	✗	✓	No
zkFL (Ours)	✓	✓	✓	✓	No

✓denotes full support; ✗denotes not supported.

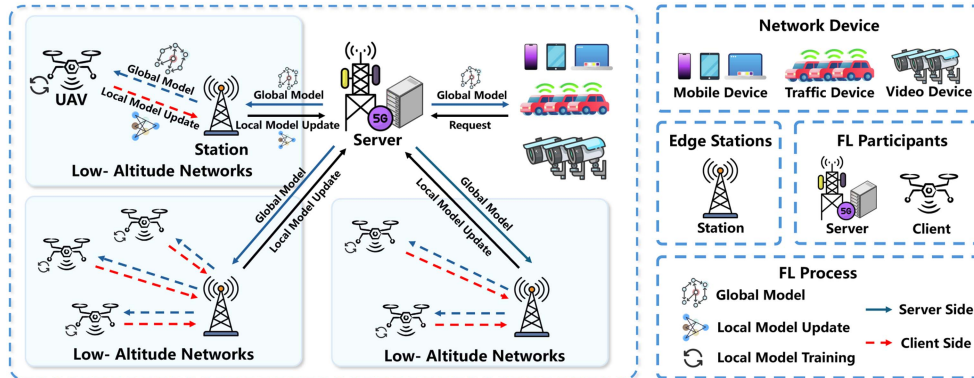


Fig. 1. Federated Learning in Low-altitude Networks.

Byzantine attacks are generally categorized into two types: untargeted and targeted [18], [20], [24], [25]. Untargeted attacks aim to globally reduce model performance across a wide range of inputs by injecting disruptive or statistically misleading updates [26]. In contrast, targeted attacks focus on corrupting the model's behavior on specific input classes or samples, enabling adversaries to enforce incorrect predictions in controlled situations [27]. To mitigate the impact of Byzantine attacks in FL, various Byzantine-robust FL methods have been proposed to enhance aggregation robustness [10], [28], [29], [30], [31], [32], [33], [34]. These methods aim to suppress the influence of

malicious updates by analyzing statistical properties, applying filtering rules, or reducing the weight of suspected clients during model aggregation. Although these methods perform well under controlled assumptions, they fail to maintain effectiveness in realistic low-altitude environments characterized by unstable communication, partial trust, and data heterogeneity. Besides, existing Byzantine-robust FL methods still suffer from several limitations as summarized in Table I.

One major limitation is that many existing methods, such as Krum [10], Trimmed Mean [28], Bulyan [29], FLTrust [30], CareFL [31], and FedInv [32], rely on the assumption that

the server is semi-honest and follows the aggregation protocol. However, when the server is malicious, it may selectively amplify harmful model updates or alter the detection process to deviate from the prescribed procedure. Such manipulations compromise the reliability of the aggregation result, allowing Byzantine clients to bypass defenses and degrade the global model without being identified. Another significant limitation lies in the inability of existing methods to accurately detect Byzantine clients in non-independent and identically distributed (non-IID) environments. In non-IID environments, benign clients may produce model updates that deviate from the majority due to inherent differences in their local data distributions. However, methods such as Krum [10], Trimmed Mean [28], and Bulyan [29] rely on assumptions of distance consistency or statistical similarity across clients. These assumptions tend to treat large deviations from the majority as signs of malicious behavior. As a result, benign clients with non-IID data are often misclassified as Byzantine clients, which reduces their contribution to model aggregation and ultimately degrades the performance of the global model.

A further limitation is the reliance on auxiliary information to support robust aggregation. For instance, FLTrust [30] and CareFL [31] require access to a clean, trusted dataset on the server to guide filtering decisions. Other methods, such as Krum [10] and Trimmed Mean [28], depend on prior knowledge of the number of Byzantine clients to correctly configure hyperparameters like the number of retained neighbors or the trimming ratio. However, in low-altitude networks, such auxiliary information is difficult to obtain due to dynamic client participation, intermittent connectivity, and the lack of stable access to trusted references.

Therefore, to achieve a robust Byzantine defense in low-altitude networks that remains effective under a potentially malicious server and does not rely on auxiliary information, three key challenges must be addressed.

- *How to accurately identify Byzantine clients when the server may behave maliciously and interfere with the detection process?*
- *How to distinguish benign and Byzantine clients under non-IID environments*
- *How to detect Byzantine behavior without relying on auxiliary information?*

Our work: In this paper, we propose zkFL, a Byzantine-robust FL framework designed to detect Byzantine clients and ensure verifiable aggregation, even in the presence of a potentially malicious server and non-IID client data. Unlike existing methods that assume a trustworthy server, zkFL enables clients to verify whether the server has followed the protocol correctly throughout each training round.

To achieve this, zkFL integrates zero-knowledge proofs into the FL process. The server must provide cryptographic proofs to demonstrate the correctness of three key steps: the use of each client's committed model update, the correctness of the aggregation weights computed from detection scores, and the consistency of the final global model with the weighted aggregation rule. These proofs enable clients to verify the integrity of the training process while preserving the confidentiality of their model updates. Besides, we observe that Membership Inference

Attacks (MIA) are highly sensitive to changes in local model behavior and can be adapted to detect Byzantine clients in non-IID environments. While benign clients tend to maintain consistent confidence distributions, Byzantine clients introduce deviations due to gradient manipulation or data poisoning. zkFL quantifies these deviations as malicious scores, which guide a dynamic server-weighted aggregation process. To prevent manipulation by a malicious server, we embed zero-knowledge proofs into the protocol, allowing clients to verify that the server has correctly used client updates, computed weights based on detection scores, and performed aggregation honestly. This joint design enables zkFL to achieve robust Byzantine defense and verifiable aggregation without relying on auxiliary datasets or trusted infrastructure.

Our main contributions can be summarized as:

- We propose zkFL, a Byzantine-robust FL framework that integrates zero-knowledge proofs to enable verifiable aggregation. Even when the server behaves maliciously, each client can locally verify whether the aggregation process complies with the declared protocol and decide whether to accept the global model for further training.
- We design a method that effectively distinguishes Byzantine behavior from natural update deviations in non-IID settings, and operates without auxiliary information such as trusted datasets, attacker ratios, or global knowledge.
- We conduct extensive experiments demonstrating that zkFL achieves strong robustness and verifiability under Byzantine attacks, and consistently outperforms state-of-the-art Byzantine-robust FL methods in global model accuracy, even when the server is malicious.

II. RELATED WORKS

In this paper, we focus on designing a Byzantine-robust FL framework with zero-knowledge proofs to defend against Byzantine attacks in low-altitude networks. This section reviews relevant background on Byzantine attacks, prior Byzantine-robust FL methods, and also discusses common decentralized FL methods under Byzantine attacks.

A. Byzantine Attack

Due to the decentralized training process, limited trust among participants, and the absence of direct verification of local computations, FL is inherently vulnerable to Byzantine attacks [10], [45], [46]. In FL, a central server aggregates updates from multiple clients under the assumption that the majority follow the prescribed protocol. However, attackers can exploit the lack of transparency and the heterogeneity of local data to submit manipulated updates. These malicious updates may degrade the global model's accuracy, hinder convergence, or introduce hidden behaviors, especially in non-IID environments [15], [18], [47]. In low-altitude network environment, such as UAV-assisted communication systems, the high mobility of nodes, dynamic network topology, and intermittent connectivity further exacerbate these vulnerabilities and amplify the impact of malicious updates on the aggregation process [48], [49], [50], [51], [52].

Byzantine attacks can be categorized according to their objectives into untargeted and targeted attacks [23], [53]. Untargeted

attacks aim to hinder convergence or reduce the overall accuracy of the global model without focusing on specific misclassification outcomes. Typical attacks include sign-flipping [54], scaling [55], and Gaussian noise injection [56], which create substantial deviations from benign updates while preserving plausible gradient structures to evade simple detection. Targeted attacks, in contrast, seek to induce specific erroneous behaviors in the global model while maintaining high accuracy on benign inputs. One typical instance is the backdoor attack [25], in which a hidden trigger causes predictable misclassification when present. Advanced targeted attacks include distributed backdoor attacks [57], which split the trigger pattern across multiple malicious clients to minimize anomalies in any single update, and sybil amplification [58], where colluding adversaries jointly amplify the impact of poisoned updates. These threats become more severe in low-altitude networks with non-IID data distributions [59], [60].

Byzantine attacks are also categorized based on their execution strategies into data poisoning and model poisoning [10], [23], [61], [62]. A data poisoning attack manipulates local training samples or labels before computing model updates, thereby embedding malicious patterns directly into the learned representations. Examples include label-flipping [63] and backdoor injection [63], which alter data semantics to bias the global model. A model poisoning attack bypasses data manipulation and directly alters gradients or parameters, enabling the adversary to craft updates that conform to aggregation rules while achieving the attack objective [62], [64]. Compared to data poisoning, model poisoning typically achieves faster convergence toward the attack goal and is harder to detect, especially in non-IID environments.

In this paper, we design a Byzantine-robust FL framework for low-altitude networks, which detects Byzantine clients and dynamically adjusts aggregation weights even under non-IID environments.

B. Byzantine-Robust Federated Learning

In FL, multiple clients collaboratively train a global model under the coordination of a central server, with each client performing local training independently and sharing only model updates [65]. The absence of direct verification of these updates makes FL vulnerable to Byzantine attacks that disrupt the training process [29]. To mitigate these threats, Byzantine-robust FL methods are developed to maintain the reliability of model training in the presence of Byzantine clients.

One class of Byzantine-robust FL methods filters out malicious model updates based on the assumption that their statistical or geometric characteristics significantly differ from those of benign updates [10], [28], [32], [66]. For example, Krum [10] and Multi-Krum [10], which select one or several updates closest to others in Euclidean space, assuming that benign updates form a dense cluster. Median and Trimmed Mean [28] compute element-wise robust statistics to suppress extreme values, while Bulyan [29] combines Multi-Krum's candidate selection with Trimmed Mean to enhance resilience. Other methods, such as FoolsGold [58], exploit historical similarity patterns or local update distributions to detect and downweight colluding

adversaries. FedInv [32] inverts client updates into dummy data and measures distributional distances to identify anomalies. Although effective in controlled settings, these methods rely on assumptions such as a known bound on the number of Byzantine clients or statistical consistency across updates [67]. In low-altitude network environments, the high mobility of aerial nodes, rapidly changing communication topology, and intermittent connectivity exacerbate non-IID data distributions and amplify statistical heterogeneity, causing benign updates to deviate substantially from the global average even without adversarial manipulation [68], [69]. Such deviations blur the distinction between honest and malicious updates, invalidate the assumption that Byzantine gradients are statistically separable, and allow carefully crafted adversarial updates to blend into the distribution of benign ones [70], [71]. Furthermore, dynamic participation and limited bandwidth hinder the accurate estimation of benign update distributions, whereas attackers may adapt their attack strategies over time to evade static statistical detection.

Another class of Byzantine-robust FL methods, filter-based methods, identify and remove potentially malicious model updates by applying predefined filtering rules before aggregation. These rules are typically derived from evaluation metrics such as accuracy, loss, or similarity measured on a trusted reference dataset [30], [31], [33] or from learned behavioral patterns over time [34], [35], [36]. Updates that fail to meet the specified criteria are either discarded entirely or assigned reduced weights in the aggregation step, thereby limiting the influence of Byzantine clients while retaining contributions from benign clients. FLTrust [30] stores a small, trusted dataset on the server to train a reference model and computes cosine similarity between each client's update and the reference gradient, discarding those with low similarity. CareFL [31] computes Shapley values from the validation loss change caused by including or excluding each client's update, enabling fine-grained, contribution-based filtering. Zeno++ [33] measures the descent score of each update, defined as its capacity to reduce the global loss on the validation dataset, and rejects those failing to meet a minimum score threshold. FoolsGold [34] tracks historical gradient similarity between clients and uses the validation data indirectly to identify colluding adversaries, penalizing their aggregation weights accordingly. FLDetector [35] predicts a client's future update based on historical behavior and compares it with the actual submission, applying unsupervised clustering on validation loss and similarity metrics to detect outliers. FLGuard [36] integrates contrastive representation learning with ensemble filtering, projecting updates into a feature space learned from validation data and combining multiple detection models to score and filter updates. FedSmart [37] periodically evaluates each client's model on the validation dataset and adjusts its aggregation weight proportionally to relative performance, reducing the impact of poisoned updates while retaining high-quality contributions.

Recent studies further explore architectural improvements or asynchronous protocols to enhance robustness. AFLGuard [41] addresses Byzantine threats in asynchronous FL by leveraging a trusted dataset to verify the direction and magnitude of delayed updates, filtering out anomalous contributions while tolerating benign staleness. BRACE [42] targets serverless

ring-all-reduce architectures, combining gradient sign quantization, subvector exchange, and majority-vote consensus to achieve communication-efficient and Byzantine-robust aggregation. FLAME [39] and EIFFeL [40] introduce temporal consistency checks across training rounds, where the evolution of client updates is monitored over time to detect abnormal drifts and suppress the influence of persistent malicious behaviors. FLGuard [36] integrates contrastive representation learning into the FL pipeline, enabling the server to learn discriminative embeddings of client updates and apply ensemble filtering during aggregation. MUDGUARD [38] combines multi-dimensional reputation tracking with adaptive defense scheduling, embedding the detection mechanism directly into the training process to respond dynamically to evolving attack strategies. Cluster-based methods, such as FedCmp [44], group client updates in the parameter space and apply majority voting within each cluster to identify and isolate malicious participants without requiring a trusted reference dataset, which improves scalability in heterogeneous environments. Meanwhile, decentralized frameworks like DFL-Dual [43] employ dual-domain distance metrics, which capture both model parameter and update direction discrepancies, and apply trust bootstrapping across peers to resist Byzantine behaviors in peer-to-peer settings, maintaining robustness even when a majority of neighboring nodes are compromised.

In addition, existing Byzantine-robust FL methods commonly assume that the server is semi-honest and strictly follows the aggregation protocol. Under this assumption, the server is responsible for collecting updates, applying defense mechanisms, and performing aggregation correctly [72], [73]. When the server itself is malicious, these methods become ineffective, as they lack the ability to detect or constrain server-side misbehavior. In this paper, we aim to design a Byzantine-robust FL method for low-altitude network environments that ensures system robustness under a potentially malicious server and enables the detection of Byzantine clients in non-IID environments without relying on trusted datasets or prior knowledge.

C. Decentralized Federated Learning

Most existing Byzantine-robust FL methods rely on a single central server to aggregate client updates, and the system fails to function properly when a single point of failure occurs. And when the server behaves maliciously, the robustness rules that safeguard against Byzantine clients become ineffective [74].

To address this limitation, decentralized Federated Learning (DFL) has been explored, where clients exchange model parameters directly without a central server, such as peer-to-peer, server-free, serverless, device-to-device, and swarm learning [75], [76], [77]. DFL removes the server as an intermediary, enabling clients to collaborate through diverse communication protocols, including point-to-point, gossip, and broadcast, under various network topologies, such as line, ring, mesh, fully connected, star, and hybrid [78], [79]. The resulting communication cost depends on the specific topology and protocol. Dense connectivity incurs substantial message exchanges, whereas sparse connectivity reduces per-client communication but limits the

ability to evaluate the reliability of distant participants. One of the primary advantages of DFL is its increased robustness. By removing the central server, the system eliminates the risk of a single point of failure (SPoF), making it more resilient to server malfunctions or malicious attacks targeting a central entity [80], [81]. Additionally, DFL enhances privacy protection, as clients only share model parameters, rather than raw data, with their peers. Furthermore, DFL can be more scalable than centralized Federated Learning, particularly in large-scale systems with numerous clients. In scenarios where clients have heterogeneous data or are spread across vast geographic regions, DFL enables more flexible and dynamic collaborations without relying on a centralized server to manage all client interactions.

However, in DFL without a central server to validate the correctness of client updates, Byzantine clients can inject incorrect or adversarial updates into the system. To address this, various Byzantine-robust DFL methods have been developed, such as gossip-based aggregation, neighbor filtering, and update clipping to ensure the integrity of model updates and safeguard against the influence of malicious clients. These methods aim to maintain the advantages of DFL while ensuring that the system remains reliable and secure in the presence of Byzantine clients. LEARN [75] employs a gossip-based strategy where clients exchange model updates and perform local aggregation using either a Minimum-Diameter Averaging or a Trimmed-Mean rule. However, as with many other DFL methods, LEARN relies on the assumption of dense connectivity between clients, which can lead to high communication costs and challenges in environments with sparse or unreliable network connectivity.

Similarly, UBAR [82] introduces a two-step neighbor filtering mechanism. Clients first select nearby neighbors based on parameter distance, then evaluate their updates using local validation loss to filter out unreliable models, achieving robustness with minimal reliance on dense topologies. This method minimizes reliance on dense network topologies but still faces challenges in ensuring robustness when the network is under extreme adversarial conditions. ClippedGossip [83] ensures consensus by clipping each neighbor's update relative to the local model and is the first to establish convergence guarantees under Byzantine failures for non-convex objectives. However, it assumes that clients can trust the local model and may struggle to maintain robustness when multiple Byzantine clients simultaneously disrupt the system. More recent methods aim to improve scalability and efficiency. For example, DFL-Dual [43] integrates dual-domain client clustering with a trust bootstrapping mechanism to iteratively exclude malicious models under extreme adversarial conditions. This method's approach of gradually refining the model aggregation process helps enhance robustness but introduces additional complexity and computational overhead.

However, despite their benefits, decentralized Byzantine-robust methods still introduce higher computational and communication overhead compared to centralized counterparts [77], [84], [85], [86]. Their distributed nature demands more sophisticated mechanisms for trust evaluation, neighbor selection, and update validation, making them less practical in resource-constrained environments.

Therefore, in this paper, we aim to achieve Byzantine-robust FL without introducing additional overhead by adopting a centralized FL framework to reduce communication costs. To ensure robustness against Byzantine attacks, especially under the presence of a malicious server, we incorporate zero-knowledge proofs (ZKPs). ZKPs allow clients to independently verify the correctness of the aggregation process, ensuring that the server adheres to the protocol without revealing sensitive model updates. This approach reduces communication overhead while ensuring the integrity of global model aggregation and protecting against malicious server behavior through the use of ZKPs.

III. PRELIMINARIES

In this section, we provide the necessary background for understanding the proposed framework. We first introduce the standard process of FL. Then, we present the basic concepts of ZKPs, which form the foundation for the verifiable mechanisms employed in our design.

A. Federated Learning

FL is a distributed machine learning paradigm that enables multiple clients to collaboratively train a global model without sharing their private data [10], [87], [88], [89], [90]. Instead of transmitting raw data, each client performs local training and sends model updates to a central server, which coordinates the aggregation process. The typical FL process can be outlined in the following steps:

Step I Initialization: The central server initializes a global model $\mathbf{w}^{(0)}$ and maintains it throughout the training process. This model is broadcast to all participating clients before training begins.

Step II Local Model Update: In each communication round t , a subset of clients is selected to participate in training. Each selected client C_i receives the global model $\mathbf{w}^{(t)}$ and performs local training on its private dataset $\mathcal{D}_i$. The client minimizes its local objective function:

$$F_i(\mathbf{w}) = \frac{1}{|\mathcal{D}_i|} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}_i} \ell(\mathbf{w}; \mathbf{x}, \mathbf{y}), \quad (1)$$

where $\ell(\cdot)$ denotes the loss function. After performing local updates, each client obtains an updated model $\mathbf{w}_i^{(t)}$ and transmits it back to the server.

Step III Global Model Aggregation: Upon receiving updates from the selected clients, the server aggregates the models according to a predefined aggregation rule. This rule assigns weights $\alpha_i^{(t)}$ to each client $C_i \in \mathcal{S}^{(t)}$ based on system-specific strategies, such as data volume, update quality, or other trust metrics. The global model is updated as follows:

$$\mathbf{w}^{(t+1)} = \sum_{i \in \mathcal{S}^{(t)}} \alpha_i^{(t)} \mathbf{w}_i^{(t)}, \quad (2)$$

where $\sum_{i \in \mathcal{S}^{(t)}} \alpha_i^{(t)} = 1$ and $\alpha_i^{(t)} \geq 0$ for all i . The updated global model $\mathbf{w}^{(t+1)}$ is then broadcast to clients in the next round, and the process repeats until convergence.

B. Hash Functions

In this paper, we adopt a cryptographic hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ that maps inputs of arbitrary length to outputs of fixed length λ . The hash function H is assumed to be collision-resistant, meaning it is computationally infeasible for any probabilistic polynomial-time adversary to find two distinct inputs $x \neq x'$ such that $H(x) = H(x')$. This property is crucial for ensuring the integrity and binding of committed model updates and auxiliary values in the proof process [91], [92], [93].

C. Commitments

To protect the confidentiality of model updates and enable verifiable aggregation, we adopt a commitment method that allows clients to commit to their local model updates without revealing them. Specifically, we use the Pedersen commitment, which provides both binding and hiding properties under standard discrete logarithm assumptions [94].

Given a local model update w_i and a randomly chosen blinding value r_i , the commitment is computed as:

$$\text{Com}(w_i, r_i) = g^{w_i} \cdot h^{r_i} \quad (3)$$

where g and h are public generators of a cyclic group with hidden order. The hiding property ensures that the committed value w_i remains secret without revealing r_i , while the binding property prevents clients from changing the committed value once it is submitted. In our framework, this commitment is later used as the input to zero-knowledge proofs for verifying the correctness of aggregation and client-side scoring under privacy constraints [95], [96].

D. Zero-Knowledge Proofs

ZKPs are cryptographic protocols that allow one party (i.e., the prover) to convince another party (i.e., the verifier) that a given statement is true without revealing any information beyond the validity of the statement itself [97], [98], [99], [100]. A ZKP protocol satisfies the following three properties:

- *Completeness:* If the statement to be proven is true, then an honest prover (i.e., one who follows the protocol) can convince an honest verifier.
- *Soundness:* If the statement is false, then a cheating prover has only a negligible probability of convincing an honest verifier that it is true.
- *Zero-knowledge:* If the statement is true, then the verifier learns nothing beyond this fact during the process.

In this work, we adopt Groth16, a widely used zk-SNARK (Succinct Non-Interactive Argument of Knowledge) construction that enables efficient proofs with minimal communication overhead [101], [102]. Groth16 is defined over a finite field $\mathbb{F}$ and is designed to prove the satisfiability of arithmetic circuits. Given a circuit $C : \mathbb{F}^n \times \mathbb{F}^h \rightarrow \mathbb{F}^l$, the prover aims to convince the verifier that for an input $x \in \mathbb{F}^n$, there exists a witness $\text{wit} \in \mathbb{F}^h$ such that:

$$C(x, \text{wit}) = 0^l \quad (4)$$

This relation defines the valid instance-witness pairs as:

$$R_C = \{(x, \text{wit}) \in \mathbb{F}^n \times \mathbb{F}^h \mid C(x, \text{wit}) = 0^l\} \quad (5)$$

And the associated language is:

$$\mathcal{L}_C = \{x \in \mathbb{F}^n \mid \exists \text{wit} \in \mathbb{F}^h \text{ such that } C(x, \text{wit}) = 0^l\} \quad (6)$$

Groth16 consists of three core algorithms:

- $(pk, vk) \leftarrow \text{KeyGen}(1^\lambda, C)$: This algorithm takes a security parameter 1^λ and the circuit C to output a proving key pk and a verification key vk .
- $\pi \leftarrow \text{Prove}(pk, x, \text{wit})$: Given pk , an input x , and a witness wit , this algorithm generates a non-interactive proof π .
- $\text{True/False} \leftarrow \text{Verify}(vk, \pi, x)$: The verifier uses vk and proof π to check whether $x \in \mathcal{L}_C$. It returns True if the proof is valid, otherwise False.

Groth16 proofs are succinct, with constant-size proofs and verification complexity that scales linearly with the size of the public input x , i.e., $O_\lambda(|x|)$. These properties make Groth16 particularly suitable for FL scenarios in low-altitude networks, where communication is constrained and real-time verification is critical. By applying Groth16, clients can efficiently prove the correctness of local computations or verify the behavior of the server without revealing sensitive model information.

IV. PROBLEM STATEMENT

We aim to achieve a Byzantine-robust FL framework that enables reliable detection of Byzantine attacks in low-altitude network environments, even when the central server behaves malicious. In this section, we first present the system model that reflects the characteristics of low-altitude FL deployments. Then, we introduce the threat model, which captures both Byzantine clients and an malicious server. Finally, we summarize the design goals.

A. System Model

FL in low-altitude network environments involves a central server and multiple aerial clients [103], [104]. The server is usually deployed at a ground control station or an edge computing node, which is responsible for broadcasting the global model, collecting local updates, and performing aggregation. The clients are typically aerial vehicles such as UAVs, which capture sensory data (e.g., images, flight trajectories, or environmental measurements) during flight missions and locally train their models before transmitting updates back to the server. Moreover, different UAVs collect distinct types of data depending on their tasks and environments, which results in data heterogeneity and non-IID characteristics [70]. These characteristics significantly increase the vulnerability of FL systems to Byzantine attacks, especially when malicious updates are introduced by compromised UAVs or when the server itself deviates from the aggregation protocol.

In this paper, we focus on detecting Byzantine behavior in FL systems deployed in low-altitude networks, where both the clients and the server may be malicious.

- *Server*: In low-altitude network environments, the server usually corresponds to a ground control station or an edge

computing node, which benefits from relatively stable connectivity and higher computational capacity. The server receives local model updates from aerial clients, aggregates them into a global model, and then redistributes the updated model back to the clients.

- *Client*: Each client corresponds to an aerial node, such as a UAV, equipped with onboard computing capability and holding a private local dataset collected during flight operations. Clients participate in training by performing local model updates based on their non-IID data and transmitting these updates to the server through wireless links. The server then aggregates the received updates into a global model and redistributes it to the clients. Upon receiving the global model, each client continues local training on its private dataset, thereby refining the model to better capture its specific data distribution before the next communication round.

B. Threat Model

In our threat model, we consider both Byzantine clients and a malicious server. Any client may become Byzantine due to malicious intent, system compromise, or local computation errors. The server may behave malicious as a result of external compromise, improper deployment, or intentional deviation from the protocol. We assume that up to $n-1$ clients in the system can act as Byzantine clients, and a single server may be malicious.

- *Byzantine Clients*: A subset of clients may behave arbitrarily due to system compromise or computation faults. These clients may submit manipulated model updates to degrade the global model's performance, prevent convergence, or inject targeted backdoors. In our setting, Byzantine clients may change their behavior across rounds, alternating between benign and malicious updates to evade detection. We assume that up to $n - 1$ clients in the system can be Byzantine.
- *Malicious Server*: The server may deviate from the protocol either due to malicious intent or external compromise. It may tamper with client updates, assign incorrect aggregation weights, forge update commitments, or provide inconsistent aggregation results. Such behaviors compromise the trust and correctness of the global model. We assume that there is at most one server in the system, and it may act semi-honestly or fully maliciously.

C. Design Goals

We aim to propose a Byzantine-robust FL framework that integrates zero-knowledge proofs to enable verifiable detection of Byzantine behavior, even when the server is potentially malicious. By embedding cryptographic proofs into the model aggregation process, our system allows each client to verify whether the server has followed the aggregation protocol faithfully, while maintaining robustness against dynamic client-side attacks in non-IID settings. And it not rely on auxiliary information and is suitable for deployment in resource-constrained low-altitude networks.

Specifically, our design goals are as follows.

- *Verifiable robustness under the malicious server:* Our method needs to allow each client to verify whether the server follows the aggregation protocol correctly, including the use of submitted updates and computation of weights.
- *Accurate detection of Byzantine clients in non-IID environments:* Our method needs to distinguish malicious clients from honest ones whose updates deviate due to non-IID data, avoiding misclassification and ensuring reliable aggregation.
- *Independence from auxiliary information:* Our method needs to work without relying on trusted datasets, attacker ratios, or fixed assumptions, making it suitable for dynamic and partially trusted FL deployments.

V. METHODOLOGY

In this section, we first introduce an overview and the design ideas of zkFL. Next, we present the design details of zkFL.

A. System Overview

Existing Byzantine-robust FL methods typically assume a semi-honest server, which leaves the system vulnerable when the server behaves maliciously (e.g., manipulating client updates or aggregation weights). Besides, these methods often rely on strong assumptions, such as access to clean reference data or known bounds on attackers, and struggle to distinguish malicious updates from benign but diverse ones in non-IID settings. To overcome these limitations, we propose zkFL, a verifiable FL framework that embeds zero-knowledge proofs into each training round. zkFL enables clients to verify whether the server correctly uses committed gradients and detection-based weights during aggregation, ensuring robustness against Byzantine attacks without relying on auxiliary trust or unrealistic assumptions.

Idea: Existing Byzantine-robust FL methods primarily focus on identifying malicious clients while assuming that the server behaves semi-honestly. However, when the server itself is malicious, such as manipulating client updates or aggregation weights, these methods can no longer ensure Byzantine robustness, especially in non-IID environments with dynamic adversaries. The lack of verifiability in the aggregation process leaves the system exposed to subtle server-side manipulations that are difficult for clients to detect or challenge. Thus, we propose zkFL, a verifiable FL framework that considers the server as potentially malicious and enforces aggregation correctness through cryptographic guarantees. In each round, zkFL embeds a zero-knowledge proof to convince all clients that the server has faithfully used the committed client gradients, computed the aggregation weights according to the detection scores, and correctly performed the global model update. This design ensures that clients can independently verify the aggregation process without relying on trust assumptions, enabling robust FL in the presence of both Byzantine clients and a malicious server.

The workflow of zkFL is as follows:

Step I. Initialization: At the beginning of training, the server initializes the global model and generates a set of public parameters, including elliptic curve generators for gradient commitment and a proving-verifying key pair for zero-knowledge proofs. The

server also prepares a dataset for Byzantine detection, which may come from public sources or synthetic generation, without requiring it to be clean or trusted. This dataset is used to pretrain a detection model that helps assess client behavior in later rounds.

Step II. Local Training and Detection: Each client receives the global model and detection dataset, then trains locally on both its private data and the detection data. After training, the client computes a commitment to its gradient using homomorphic commitment and submits the gradient, randomness, and commitment to the server. The server evaluates the uploaded models on the detection data using a pre-trained discriminator and computes a detection score for each client, which reflects the likelihood of benign or malicious behavior.

Step III. Verifiable Aggregation: Based on the detection scores, the server dynamically assigns aggregation weights and computes the global model update using a weighted sum of client gradients. To ensure the correctness of this process, the server constructs a zero-knowledge proof that demonstrates the validity of each client's commitment, the correctness of weight computation, and the integrity of the aggregation. The updated model, commitments, and the proof are then broadcast to all clients for independent verification.

We illustrate the overall workflow of zkFL in Fig. 2. In the following, we present the detailed design of the commitment mechanism, Byzantine client detection, and the construction of zero-knowledge proofs.

B. Gradient Commitment Mechanism

At the beginning of training, the server initializes the global model w_0 with random or pre-trained parameters. It also executes the zk-SNARK setup algorithm

$$(pp, vk) \leftarrow \text{Setup}(1^\lambda, C), \quad (7)$$

where C denotes the arithmetic circuit that constrains the aggregation computation. The proving key pp is kept by the server for proof generation, while the verification key vk is distributed to all clients for proof verification. In addition, the server generates a set of public elliptic curve generators $\{G_j\}_{j=1}^d$ and h via a hash-to-curve function, which are used for Pedersen commitments of gradient vectors. These public parameters $(vk, \{G_j\}, h)$ and the initial global model w_0 are broadcast to all participating clients before the first training round.

To prevent the server from tampering with client-submitted gradients during aggregation, zkFL requires each client to generate a homomorphic commitment to its local update. This commitment binds the submitted gradient to a public representation without revealing the gradient content, and it supports additive homomorphism, which is essential for subsequent proof construction.

Let the local (real-valued) gradient of client i be

$$\mathbf{g}_i = (g_{i1}, \dots, g_{id}) \in \mathbb{R}^d, \quad (8)$$

and let each coordinate be encoded into $\mathbb{Z}_p$ via a fixed-point map with a public scaling factor, which we denote by the same symbols for simplicity. The client samples a random scalar $r_i \in$

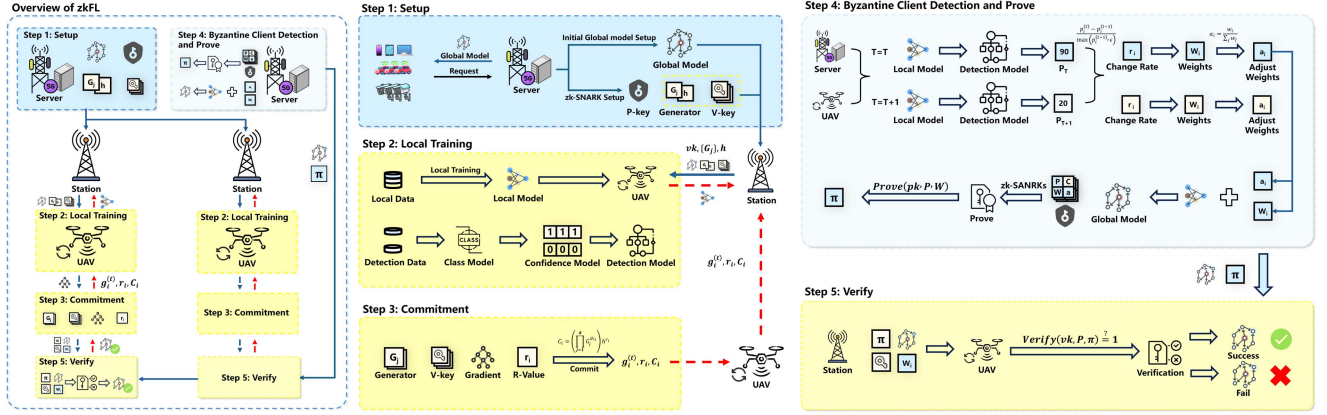


Fig. 2. Overview of the proposed Zero-knowledge proof Byzantine-robust Federated Learning method, zkFL in low-altitude network.

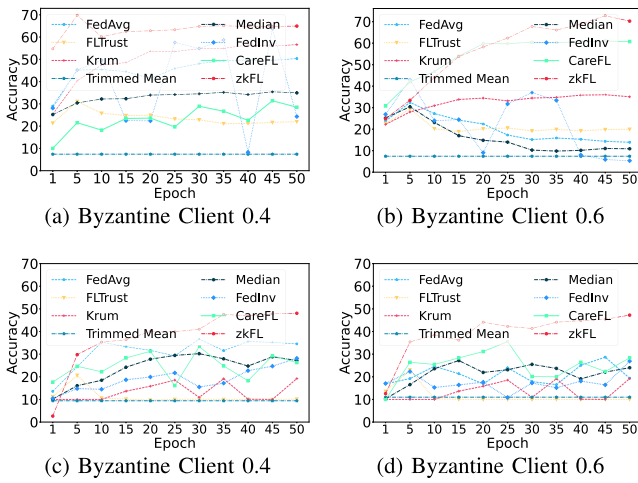


Fig. 3. Accuracy comparison and performance trend between zkFL and other methods on the CIFAR dataset under both IID (a) and (b) and non-IID (c) and (d) settings.

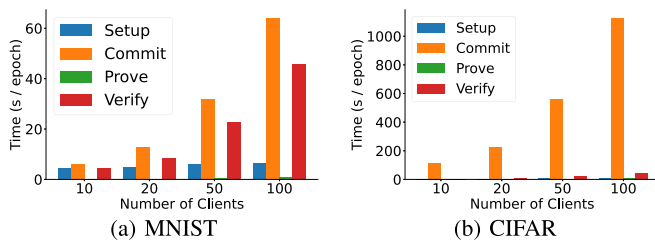


Fig. 4. Time breakdown comparison of zkFL on the MNIST dataset (a) and the CIFAR dataset (b), including the setup, commit, prove, and verify phases across different client numbers.

$\mathbb{Z}_p$ and computes the commitment

$$C_i = \left(\prod_{j=1}^d G_j^{g_{ij}} \right) h^{r_i}, \quad (9)$$

where $\{G_j\}_{j=1}^d$ and h are fixed public generators on an elliptic curve (e.g., BLS12-381), derived via a hash-to-curve function during initialization. Once published, the commitment is binding (under the discrete-log assumption relative to the basis $\{G_j\}$, h) and hiding (the committed value is information-theoretically

concealed given uniformly random r_i). The construction is additively homomorphic, enabling aggregation in the commitment space consistent:

$$C_{\text{sum}} = \prod_i C_i = \left(\prod_i \prod_{j=1}^d G_j^{g_{ij}} \right) h^{\sum_i r_i}. \quad (10)$$

Although g_i may be sent in plaintext for computing aggregation weights, only the commitments and the associated zk-SNARK proofs are involved in verification. In stronger threat models, the plaintext gradient does not need to be disclosed. Instead, clients can keep g_i hidden and prove in zero knowledge that the committed value C_i is consistent with their local update, without revealing the gradient itself.

C. Byzantine Client Detection

To enable Byzantine client detection, the server first trains two models on its own dataset prior to the federated training process. A classification model M_{class} is trained to distinguish detection data from other data, and then used to extract confidence features from model outputs. These features are labeled and used to train a discriminator model M_{det} , which serves as the basis for scoring client behavior during training.

To detect Byzantine behavior in a non-IID environment without relying on clean datasets or known attacker ratios, zkFL adopts an inference-guided scoring mechanism. The server first uses a pre-trained classification model M_{class} to simulate benign client predictions on a designated detection dataset $\mathcal{D}_{\text{det}}$. Then, for each client model M_i , the server computes its output on $\mathcal{D}_{\text{det}}$, and the resulting confidence vectors are evaluated by a discriminator model M_{det} to produce a score

$$p_i^{(t)} \in [0, 1]. \quad (11)$$

A high score indicates that the client likely followed the prescribed protocol; a low score suggests potential Byzantine behavior. To handle dynamic behaviors, the server further calculates the rate of change between consecutive rounds as

$$r_i = \frac{p_i^{(t)} - p_i^{(t-1)}}{\max(p_i^{(t-1)}, \varepsilon)}, \quad (12)$$

Algorithm 1: Byzantine Client Detection in zkFL.

Server Input: Global model M_{global} , detection dataset $D_{\text{in-det}}$, non-detection dataset $D_{\text{out-det}}$, discriminative model M_{det} , decision threshold θ , weight thresholds $\theta_{\text{pos}}, \theta_{\text{neg}}$, learning rates $\alpha_{\text{recover}}, \alpha_{\text{increase}}$

Client Input: Local dataset and $D_{\text{in-det}}$

Output: Detection score $p_i^{(t)}$ and aggregation weight α_i for each client

```

1 for  $t = 1$  to  $T$  do
2   Central server executes:
3   if  $t=1$  then
4     Train classification model  $M_{\text{class}}$  on  $D_{\text{in-det}}$ 
      and  $D_{\text{out-det}}$ ;
5      $D_{\text{in-features}} = \{M_{\text{class}}(x) \mid x \in D_{\text{in-det}}\}$ ;
6      $D_{\text{out-features}} = \{M_{\text{class}}(x) \mid x \in D_{\text{out-det}}\}$ ;
7     Train  $M_{\text{det}}$  using:
8      $D_{\text{in-features}}$  labeled as 1,
9      $D_{\text{out-features}}$  labeled as 0;
10  end
11  Client executes:
12  for each client  $i$  do
13    Train local model update  $M_i$ ;
14  end
15  Central server executes:
16  for each client  $i$  do
17     $F_i = \{M_i(x) \mid x \in D_{\text{in-det}}\}$ ;
18     $\text{Probs}_i = \{M_{\text{det}}(f) \mid f \in F_i\}$ ;
19     $p_i^{(t)} = \text{Mean}(\text{Probs}_i \geq \theta)$ ;
20     $r_i = \frac{p_i^{(t)} - p_i^{(t-1)}}{\max(p_i^{(t-1)}, \varepsilon)}$ ;
21    if  $r_i < \theta_{\text{neg}}$  then
22       $w_i = 0$ ;
23    else if  $r_i > \theta_{\text{pos}}$  then
24       $w_i = \min(w_i + \alpha_{\text{recover}}, 1.0)$ ;
25    else
26       $w_i = \min(w_i + \alpha_{\text{increase}}, 1.0)$ ;
27    end
28  end
29  Normalize weights:  $\alpha_i = \frac{w_i}{\sum_j w_j}$  for all clients;
30 end

```

where ε is a small constant to avoid division by zero. This rate reflects the client's behavioral shift. Based on r_i , the server assigns a raw weight w_i . If the score drops below a negative threshold θ_{negative} , the weight is reset to zero. If the score rises above a positive threshold θ_{positive} , the weight is increased incrementally. Otherwise, the weight is slowly increased unless the detection score deviates significantly from the trimmed mean, in which case the weight is also reset to zero. The final normalized aggregation weight is given by

$$\alpha_i = \frac{w_i}{\sum_j w_j}. \quad (13)$$

This mechanism ensures that clients exhibiting unstable or abnormal behavior are gradually excluded from aggregation, while benign but statistically diverse clients in non-IID settings are retained. All intermediate variables including $p_i^{(t)}$, r_i , w_i , α_i are later included in the zero-knowledge proof.

Algorithm 2: Verifiable Aggregation via Zero-Knowledge Proof.

Server Input: Previous model $\mathbf{w}_t$, gradients $\{\mathbf{g}_i\}$, randomness $\{r_i\}$, commitments $\{C_i\}$, weights $\{\alpha_i\}$, learning rate η , detection scores $\{p_i^{(t)}, p_i^{(t-1)}\}$, proving key pk

Client Input: Gradient $\mathbf{g}_i$, randomness r_i , generators $\{G_j\}$ and h , verifying key vk , received proof π_t , model $\mathbf{w}_{t+1}$, commitments $\{C_i\}$, scores $\{p_i^{(t)}, p_i^{(t-1)}\}$, weights $\{\alpha_i\}$

Output: Verification result and model acceptance

```

1 for  $t = 1$  to  $T$  do
2   Each client executes:
3   Generate commitment:
4      $C_i = \left(\prod_{j=1}^d G_j^{g_{ij}}\right) h^{r_i}$ ;
5   Send  $(\mathbf{g}_i, r_i, C_i)$  to server;
6   Central server executes:
7   Compute global update:
8      $\mathbf{w}_{t+1} = \mathbf{w}_t + \eta \sum_i \alpha_i \mathbf{g}_i$ ;
9   Construct public input:
10   $P = \{\mathbf{w}_t, \mathbf{w}_{t+1}, \eta, \{C_i\}, \{p_i^{(t)}, p_i^{(t-1)}\}, \{\alpha_i\}\}$ ;
11  Construct witness:
12     $W = \{\{\mathbf{g}_i\}, \{r_i\}\}$ ;
13  Generate proof:
14     $\pi_t = \text{Prove}(\text{pk}, P, W)$ ;
15  Broadcast  $\pi_t$ ,  $\mathbf{w}_{t+1}$ ,  $\{\alpha_i\}$ , and  $\{C_i\}$  to clients;
16  Each client executes:
17  Receive broadcast from server;
18  Reconstruct  $P$  from received data;
19  Run proof verification:
20     $\text{ok} = \text{Verify}(\text{vk}, P, \pi_t)$ ;
21  if  $\text{ok} = 1$  then
22    Accept  $\mathbf{w}_{t+1}$  as valid model update;
23  end
24  else
25    Reject update and skip local model replacement;
26  end
27 end

```

D. Verifiable Aggregation Via Zero-Knowledge Proof

To ensure the integrity of the aggregation process under an untrusted server, zkFL integrates a Groth16-based zero-knowledge proof that allows clients to verify that the server has strictly followed the aggregation protocol. The proof attests that the server uses gradients committed by the clients, applies the previously computed weights $\{\alpha_i\}$, and generates the global model update accordingly.

Since the per-client behavior change rate r_i and the normalized aggregation weight α_i are already computed in the detection and weighting phase, the zk-SNARK circuit does not re-derive them but instead checks whether the server's claimed values conform to the expected consistency constraints. Each client i submits a gradient vector $\mathbf{g}_i \in \mathbb{R}^d$, a scalar randomness $r_i \in \mathbb{Z}_p$, and a commitment C_i , which binds the gradient via a

homomorphic encoding as

$$C_i = \left(\prod_{j=1}^d G_j^{g_{ij}} \right) h^{r_i}, \quad (14)$$

where $\{G_j\}_{j=1}^d$ and h are fixed public generators defined over an elliptic curve. Specifically, the zk-SNARK circuit enforces three classes of constraints. First, for each client i , it checks that the claimed gradient $\mathbf{g}_i$ and randomness r_i are consistent with the Pedersen commitment C_i . Second, it validates that the normalized aggregation weights $\{\alpha_i\}$ correspond to the detection and weighting phase, ensuring that the server cannot substitute arbitrary coefficients. Third, it verifies the correctness of the global update by enforcing the arithmetic relation

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \eta \sum_i \alpha_i \mathbf{g}_i. \quad (15)$$

These constraints guarantee that the update is computed exactly from the committed inputs and published weights. The proof's public input P includes the current and next-round global model parameters $\mathbf{w}_t, \mathbf{w}_{t+1}$, the learning rate η , the client commitments $\{C_i\}$, the per-client detection scores $\{p_i^{(t)}, p_i^{(t-1)}\}$, and the computed aggregation weights $\{\alpha_i\}$. The private witness W contains the set of original gradients $\{\mathbf{g}_i\}$, the corresponding randomness values $\{r_i\}$, and optionally the raw weights $\{w_i\}$, which can also be excluded if already validated externally. The server invokes the Groth16 prover to generate a zero-knowledge proof

$$\pi_t = \text{Prove}(pk, P, W), \quad (16)$$

and broadcasts $\pi_t, \mathbf{w}_{t+1}, \{\alpha_i\}$, and $\{C_i\}$ to all participating clients. Upon receiving this broadcast, each client reconstructs the public input and runs

$$\text{Verify}(vk, P, \pi_t) \quad (17)$$

using the public verification key vk . If the verification passes, the client accepts the update and replaces its local model with $\mathbf{w}_{t+1}$; otherwise, the client rejects the server's result, as the server has failed to prove that the committed gradients and published weights were honestly used in aggregation. The zkFL protocol employs Groth16, which offers constant-size proofs and efficient verification. The size of π_t remains fixed regardless of the circuit size, ensuring stable communication overhead, while the verification time grows only linearly with the number of public inputs $|P|$. Proof generation complexity depends on the number of circuit constraints, but since the aggregation circuit in zkFL mainly encodes linear relations among gradients, commitments, and weights, the overall constraint size remains moderate. As a result, the prover can construct π_t with predictable cost, and each client only needs to perform a small number of bilinear pairing checks to validate the proof. These properties together ensure that the additional overhead introduced by verifiability remains bounded across training rounds. While such proofs guarantee the correctness of aggregation at the cryptographic level, zkFL further complements this mechanism with behavioral verification of weight assignment.

Moreover, clients retain the ability to verify the fairness of weight assignment beyond the cryptographic proof. Given

access to the detection dataset $\mathcal{D}_{\text{det}}$, a client may recompute its own detection score $p_i^{(t)}$, estimate the expected r_i , and reapply the thresholding rule to obtain a local estimate of its aggregation weight. This behavioral verification can be used to assess whether the server's published weights align with the expected detection logic, even when the zk proof itself passes. If a client identifies inconsistencies between expected and received weights, it may choose to abstain from future aggregation rounds, log the discrepancy, or coordinate with other clients for further auditing. This dual-layer defense ensures that the aggregation is not only cryptographically correct but also aligned with the detection semantics, thus providing strong accountability in FL under a malicious server.

VI. THEORETICAL ANALYSIS

In the following, we present theoretical analyses to demonstrate that zkFL achieves verifiable security against malicious servers and Byzantine clients, while preserving privacy and introducing only limited overhead to the overall training efficiency in FL.

A. Security Analysis

In this section, we formally analyze the security guarantees of zkFL. We prove that zkFL provides aggregation verifiability, robustness against Byzantine clients, and privacy preservation of honest clients' local gradients, even when the server is malicious and colludes with a subset of clients. In particular, we emphasize that the observable parameters during each round, including commitments, aggregation weights, and detection scores, do not reveal private information about honest clients' datasets or gradients.

Theorem 1: (Security of zkFL) Assume the presence of a malicious server and up to $f < N$ Byzantine clients among a total of N clients $\{C_1, \dots, C_N\}$. Under the zkFL protocol, each training round proceeds with clients uploading their local gradients $\{g_i\}$ along with declared aggregation weights $\{\alpha_i\}$, and the server publishing a zk-SNARK proof π_t certifying correct aggregation. Then, zkFL ensures that all honest clients can verify whether the server faithfully executes the aggregation rule, that the global model is resistant to Byzantine corruption via dynamic weighting, and that publicly observable parameters (e.g., commitments, weights, detection scores, and proofs) do not leak private information of honest clients.

We prove Theorem 1 by establishing the following two lemmas.

Lemma 1: Suppose all clients are honest and the server is malicious. Then, for each client C_i , there exists a simulator S_1 such that:

$$\text{View}_{C_i}^{\text{real}} \approx_c S_1(D_i, g_i, \alpha_i, M) \quad (18)$$

That is, zkFL guarantees that each honest client can verify whether the server faithfully executed the aggregation rule, without learning the gradients of other clients.

Proof: In zkFL, the server is required to provide a zk-SNARK proof π_t for each round, attesting that the model update satisfies $M_{t+1} = M_t - \eta \cdot \sum_{j=1}^N \alpha_j^{(t)} g_j^{(t)}$. The correctness of π_t implies that the aggregation strictly follows the protocol using

the committed local gradients and publicly declared weights. Since each client knows its own g_i and α_i , and the commitments and proof are publicly observable, a simulator $\mathcal{S}_1$ can reconstruct the view of any honest client without accessing other clients' private data. Moreover, although commitments $\{C_j\}$, weights $\{\alpha_j\}$, and proofs π_t are visible, they do not reveal sensitive information: Pedersen commitments are information-theoretically hiding with respect to randomness, and zk-SNARKs are zero-knowledge, ensuring that only correctness of aggregation is revealed but not raw gradient values.

Lemma 2: Suppose the server is semi-honest and colludes with a subset $\mathcal{B} \subset \{\mathcal{C}_1, \dots, \mathcal{C}_N\}$ of up to f Byzantine clients. Then there exists a simulator $\mathcal{S}_2$ such that:

$$\text{View}_{\mathcal{S}, \mathcal{B}}^{\text{real}} \approx_c \mathcal{S}_2(\{D_k\}_{k \in \mathcal{B}}, \{g_k\}_{k \in \mathcal{B}}, \{\alpha_k\}_{k \in \mathcal{B}}, M) \quad (19)$$

That is, the view of the colluding server and clients can be simulated without accessing the gradients or randomness of honest clients.

Proof: Each client commits to its local gradient g_j using a binding commitment $C_j = \text{Com}(g_j, r_j)$, where the randomness r_j is kept secret. The server and colluding clients observe only the public commitments, declared weights, and the zk-SNARK proof π_t , which attests to the correct use of committed values in the aggregation. The hiding property of commitments guarantees that gradients cannot be extracted from commitments without the randomness, while the zero-knowledge property of zk-SNARKs ensures that the generated proofs reveal nothing beyond the validity of the aggregation. In particular, zero-knowledge means that for any proof π_t attesting to the correctness of $M_{t+1} = M_t - \eta \cdot \sum_{j=1}^N \alpha_j^{(t)} g_j^{(t)}$, there exists a simulator that can produce an indistinguishable proof without knowledge of the actual gradients. Thus, even though π_t is published to all clients, it conveys only the fact that the aggregation was performed correctly, and leaks no additional information about the underlying gradient values or local datasets. Therefore, a simulator $\mathcal{S}_2$ can generate the entire view of the adversary without any access to honest clients' internal information. ■

By combining Lemma 1 and Lemma 2, we conclude that zkFL achieves secure aggregation verification, Byzantine robustness, and strong privacy guarantees against a malicious server and colluding clients. The design of zkFL ensures that the server's behavior is publicly auditable through zk-SNARK proofs, and the hiding property of commitments together with the zero-knowledge property of zk-SNARKs guarantee that adversaries cannot extract or infer sensitive information from honest participants' gradients or datasets. In particular, zero-knowledge ensures that any published proof is indistinguishable from one generated by a simulator without access to private data, thereby eliminating the risk of information leakage from the verification process itself.

B. Efficiency Analysis

In this section, we analyze the efficiency of zkFL in terms of computational complexity and communication overhead.

1) *Time Overhead:* The client-side overhead mainly arises from local training and zk-SNARK proof generation. Assuming E local epochs over a dataset of size $|D_k|$, and $f(M)$ as the per-sample training cost of model M , the local training cost is $O(E \cdot |D_k| \cdot f(M))$.

Proof generation introduces an additional complexity C_{zk} , which depends on the proof system and model architecture. Thus, the total client-side complexity per round is $O(E \cdot |D_k| \cdot f(M) + C_{zk})$.

On the server side, the dominant operations include proof verification and aggregation. Let V_{zk} denote the cost of verifying a zk-SNARK proof. The total server-side cost is approximately:

$$O(N \cdot V_{zk} + N \cdot d) \quad (20)$$

where d is the dimensionality of model updates. Since Groth16 proofs are constant in size, verification requires only a small number of bilinear pairing operations, and the verification complexity grows linearly with the number of clients N . Therefore, the additional cost introduced by zkFL remains predictable and bounded in each training round.

2) *Communication Overhead:* Each client uploads its model update and a zk-SNARK proof. Assuming model update size α and proof size β , the total upload cost per client per round is $O(\alpha + \beta)$.

zkFL avoids repeated communication rounds due to its non-interactive design. While the proof incurs extra bandwidth overhead, it eliminates the need for additional trust or auxiliary channels, offering a one-shot verifiable update mechanism. Since β is constant (a few hundred bytes in Groth16), the communication overhead is dominated by the model update size α . This ensures that the overall bandwidth cost of zkFL remains comparable to standard FL, with only a small constant-factor increase.

Overall, zkFL introduces additional computation and bandwidth compared to standard FL, but both time and communication overhead remain within a controllable range, while achieving strong security and robustness guarantees under realistic adversarial assumptions.

VII. EXPERIMENTS

A. Experimental Setup

1) *Environment of Experiments:* Our experiments were conducted on a server running Ubuntu 20.04 with Python 3.8 and PyTorch 1.11.0. The system is equipped with a single NVIDIA RTX 3080 GPU (10GB VRAM), 12 vCPUs from an Intel(R) Xeon(R) Platinum 8255C CPU @ 2.50GHz, and 40GB of RAM. The CUDA version is 11.3.

2) *Dataset:* We evaluated zkFL using an image recognition dataset, MNIST [105] and CIFAR-10 [106] dataset. **MNIST** dataset contains handwritten digit images for the digits 0 - 9, including 60,000 training samples and 10,000 test samples, each image being a 28x28 pixel square. **CIFAR-10** dataset consists of 60,000 32x32 color images across 10 object classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck), with 50,000 training samples and 10,000 test samples.

We employed two methods to partition the dataset for each client: (I) For the IID setting, we randomly shuffled each dataset and distributed samples equally to all clients. (II) For the non-IID setting, we partitioned MNIST into 200 shards of 300 samples each and CIFAR-10 into 200 shards of 250 samples each, with each client assigned two randomly selected shards.

3) *Model and Hyperparameter Settings*: In our experimental setup, we trained Convolutional Neural Networks (CNNs) on both the MNIST and CIFAR-10 datasets to evaluate model performance under IID and non-IID scenarios. For MNIST, we adopted a standard CNN consisting of two convolutional layers followed by max-pooling and three fully connected layers. For CIFAR-10, we used a deeper CNN tailored to the higher-dimensional input: the first convolutional layer applies 64 filters of size 5×5 , followed by a max-pooling layer with kernel size 3 and stride 2; the second convolutional layer also uses 64 filters of size 5×5 with the same pooling configuration. The flattened output is then fed into three fully connected layers and the final output is normalized through a log-softmax function. For local training, the learning rate was set to 0.001, the local epochs to 5, the SGD momentum to 0.5, and the batch size to 10. In the FL, the server selected all clients in each round and varied the proportion of Byzantine clients to 20%, 40%, and 50% to assess the robustness of the system. Each client was initialized with the same parameters, and local training was performed independently on its private dataset as well as the server-distributed datasets, reflecting the heterogeneity of data in non-IID settings. Users can adjust $\theta_{positive}$ and $\theta_{negative}$ as needed. In this experiment, to simulate real-world scenarios with an unknown number of Byzantine clients, $\theta_{positive}$ and $\theta_{negative}$ are set to the 85th and 15th percentiles. A higher $\theta_{positive}$ and lower $\theta_{negative}$ improve Byzantine client exclusion but may also exclude benign clients. The decision threshold θ is set to 0.5, as the detection model is trained using a server dataset divided into two categories.

4) *Baseline*: We used the following aggregation rules as baselines in our experiments.

- *FedAvg* [21]: Each client trains a local model on its data and sends it to the server, which averages the local models to update the global model.
- *FLTrust* [30]: The server computes a trusted update from verified clients and integrates only updates closely aligned with this trusted baseline into the global model.
- *Krum* [10]: Krum selects the client update with the smallest sum of distances to the $n - f - 2$ closest updates, where n is the total number of clients and f is the estimated number of Byzantine clients.
- *Trimmed Mean* [28]: The server trims a set proportion of the highest and lowest values from client updates and averages the remaining values to update the global model.
- *Median* [28]: For each local model update, the server selects the median value from the sorted list of K client updates for the global model update.
- *FedInv* [32]: The server inverts client updates into a dummy dataset, a synthetic dataset created by reversing the updates, and computes the Wasserstein distance to identify and filter

TABLE II
CLIENT-SIDE COMPUTATIONAL COMPLEXITY BY STAGE

Computation Stage	Computational Complexity
Local Model Training	$O(E \cdot D_k \cdot f(M))$
ZK Proof Generation	$O(C_{zk})$
Model Update + Proof	$O(\alpha + \beta)$

E is the number of local training epochs, $|D_k|$ the local dataset size, $f(M)$ the per-sample training cost, C_{zk} the complexity of zk-SNARK proof generation, α the size of a model update, and β the size of a zk-SNARK proof.

out anomalous updates before aggregating the remaining updates to update the global model.

- *CareFL* [31]: CareFL evaluates the contribution of each client by computing Shapley values based on the reduction in global loss rather than relying on static model gradients.

5) *Evaluated Byzantine Attacks*: Representative Byzantine attacks from both untargeted and targeted categories. Untargeted attacks include the Random Gradient Attack and Random Noise Attack, which aim to disrupt model convergence without specific misclassification goals. Targeted attacks include the Label-Flipping Attack, which manipulates model predictions toward specific incorrect classes [10], [18], [63].

- *Label-Flipping Attack* [63]: Byzantine clients mislead the global model by systematically flipping class labels during training, causing targeted misclassifications.
- *Random Noise Attack* [18]: Byzantine clients add random noise to their true gradients before submission, aiming to subtly corrupt the global model update. This makes malicious behavior harder to detect and can gradually degrade model performance, especially in non-IID settings
- *Random Gradient Attack* [10]: Byzantine clients send gradients drawn uniformly from random directions, aiming to disrupt the optimization trajectory. Such updates degrade convergence and reduce global model accuracy.

6) *Performance Metrics*: (1) *Accuracy*: To evaluate the performance in defending against Byzantine attacks, we use the test accuracy of the global model on benign test data as the assessment criterion. (2) *Loss Value*: The loss value measures the global model's convergence, computed as the weighted average of local losses across clients. (3) *Time Overhead*: The time overhead reflects the average computation time per round, covering both local updates and global aggregation.

B. Evaluation of Defense Performance Against Byzantine Attack

We evaluate the performance of zkFL on the MNIST dataset under both IID and non-IID settings, with a total of 50 clients participating in the training process. Byzantine clients are randomly selected in each round and execute randomized attacks. We consider three different proportions of Byzantine clients: 40% and 60%, and compare the model accuracy of zkFL under both an semi-honest server and a malicious server. The corresponding results are reported in Table IV. To further validate the generality of our method, we compare the convergence performance of zkFL with baseline approaches on the CIFAR-10 dataset. Fig. 3 presents the accuracy curves over training rounds under

TABLE III
SERVER-SIDE COMPUTATIONAL COMPLEXITY BY STAGE

Computation Stage	Computational Complexity
Proof Verification	$O(N \cdot V_{zk})$
Model Aggregation	$O(N \cdot d)$

N is the number of clients, V_{zk} the time to verify a single zk-SNARK proof, and d the dimensionality of model updates.

Byzantine attacks. Finally, we evaluate the overhead introduced by the zero-knowledge proof components in zkFL. We measure both the time cost for each round's setup, commitment, proof generation, and verification. The results are summarized in Tables V and VI and Fig. 4, respectively.

1) *Overall Performance*: From Table IV, we observe that zkFL consistently achieves the highest accuracy across all tested Byzantine attacks in the IID setting, even under the presence of a malicious server. While many baseline methods (e.g., FedAvg, FLTrust, Krum, Trimmed, Median, FedInv, and CareFL) experience significant performance degradation when the server deviates from honest behavior, zkFL maintains stable accuracy across different attack types and Byzantine client proportions.

For example, under Label-Flipping (LF) attacks with 60% Byzantine clients and a malicious server, most existing defenses fail to maintain performance: FLTrust drops to 11.98%, Median to 9.74%, and FedInv to 13.47%. Even robust methods like CareFL only reach 13.92% in this setting. In contrast, zkFL achieves 98.42%, maintaining high accuracy despite server-side manipulation. Similar trends are observed under Random Noise (RN) and Random Gradient (RG) attacks. When 60% of the clients are Byzantine and the server is malicious, FLTrust and FedInv yield only 12.86% and 16.13% accuracy under RN attacks, respectively. However, zkFL sustains 97.68% accuracy. Under RG attacks with the same adversarial setting, zkFL still achieves 97.92%, significantly outperforming alternatives.

Overall, these results confirm that zkFL achieves verifiable robustness under both high Byzantine client ratios and malicious server behavior, outperforming state-of-the-art Byzantine-robust FL methods by a large margin. By embedding zero-knowledge proofs into the aggregation protocol, zkFL allows clients to verify that the server includes submitted updates and uses correct aggregation weights. This verifiable mechanism prevents server-side tampering and ensures reliable model updates even when the server is compromised.

2) *Robustness Against Byzantine Attack in non-IID Environment*: Under non-IID environments, zkFL consistently outperforms other defenses in both semi-honest and malicious server settings, as shown in Table IV.

For LF attacks, when 60% of clients are Byzantine and the server is malicious, FLTrust, Median, and FedInv drop to 9.43%, 9.74%, and 13.87%, respectively. These methods rely on clean reference datasets or statistical assumptions that fail under data heterogeneity. Besides, they assume the server honestly executes detection or weighting mechanisms. When the server is malicious, it can arbitrarily manipulate aggregation weights or omit benign updates, rendering these defenses ineffective. In contrast,

zkFL maintains 65.68% accuracy, demonstrating the advantage of client-verifiable aggregation and server-independent detection. For RN attacks, Byzantine clients add small perturbations to evade detection. Semi-honest assumptions allow some methods to perform modestly, but under a malicious server, these methods cannot verify whether noisy updates are properly suppressed. As a result, FedAvg, Trimmed, and FLTrust drop below 13%. In comparison, zkFL achieves 55.88%, as clients independently verify the validity of server-side aggregation and detect unreliable updates based on confidence distributions. For RG attacks, where malicious updates are entirely random, most baselines collapse under a malicious server due to the absence of verifiability such as Krum and CareFL fall to 9.60% and 12.69%, respectively. zkFL still retains 67.35% accuracy, since its detection is guided by model confidence and not vulnerable to server-side manipulation.

These results highlight that zkFL preserves robustness even when both client distributions are highly non-IID and the server is adversarial. By combining inference-guided detection with zero-knowledge-verifiable aggregation, zkFL overcomes key limitations of existing defenses that assume trust in the server or require strong prior knowledge.

However, as the number of Byzantine clients increases, zkFL's accuracy gradually declines. We consider this a normal phenomenon because in a non-IID environment, the quality of each client's dataset cannot be guaranteed. Consequently, certain benign clients may produce gradient updates that are less effective than the carefully crafted malicious gradients in complex models and datasets.

3) *Experimental Results on CIFAR Dataset*: In this experiment, we evaluate the performance of zkFL on the CIFAR dataset under the Byzantine attack scenario. During the first five training rounds, no attack is launched, and the training process proceeds normally. Starting from the 6th round, Byzantine attacks are introduced, and the server is assumed to behave maliciously.

As illustrated in Fig. 3, zkFL maintains a consistently higher accuracy than all compared methods across both IID and non-IID settings. In the IID environments, Fig. 3(a) shows the case where 40% of clients are Byzantine. Here, zkFL steadily improves accuracy across epochs, while other baselines either converge to a lower plateau (e.g., FLTrust, CareFL) or exhibit instability (e.g., Trimmed Mean). When the Byzantine ratio increases to 60%, as in Fig. 3(b), most baselines suffer a significant drop or even collapse, whereas zkFL still achieves a clear upward trend, demonstrating robustness against stronger adversarial presence. In the non-IID environments, Fig. 3(c) presents the scenario with 40% Byzantine clients. Although accuracy fluctuates due to data heterogeneity, zkFL consistently outperforms all other methods and shows a more stable convergence trend. Other defenses such as FLTrust and CareFL experience unstable convergence when the attack ratio increases, while Trimmed Mean fails to converge under this setting. At 60% Byzantine ratio in Fig. 3(D), most competing methods still exhibit convergence but with significant instability and lower accuracy, whereas zkFL sustains substantially higher accuracy throughout, confirming its resilience under both heterogeneity and stronger adversarial manipulation.

TABLE IV
ACCURACY PERFORMANCE UNDER DIFFERENT BYZANTINE ATTACKS

IID	LF attacks / %				RN attacks / %				RG attacks / %			
	0.4 Byzantine		0.6 Byzantine		0.4 Byzantine		0.6 Byzantine		0.4 Byzantine		0.6 Byzantine	
	Semi-Honest	Mali-cious	Semi-Honest	Mali-cious	Semi-Honest	Mali-cious	Semi-Honest	Mali-cious	Semi-Honest	Mali-cious	Semi-Honest	Mali-cious
FedAvg [21]	62.83	15.36	10.05	3.92	94.24	13.64	88.91	11.98	94.29	13.41	92.15	14.18
FLTrust [30]	94.36	14.81	55.10	11.98	94.24	11.39	88.91	12.86	92.32	11.01	92.15	14.10
Krum [10]	97.48	15.58	97.73	12.30	97.17	15.11	97.10	13.13	97.28	15.43	97.33	14.76
Trimmed [28]	87.03	13.39	6.80	13.45	97.34	17.71	95.94	15.90	86.67	12.10	10.32	12.92
Median [28]	97.76	9.74	44.46	9.74	97.95	9.74	31.15	9.74	96.45	9.74	90.32	9.74
FedInv [32]	97.55	14.50	48.06	13.47	96.62	13.03	94.18	16.13	97.82	14.30	30.86	13.56
CareFL [31]	98.09	85.24	97.99	13.92	56.72	10.31	60.92	13.64	97.91	15.60	97.98	13.98
zkFL (ours)	98.12	97.98	98.30	98.42	98.30	97.79	97.80	97.68	97.92	97.83	98.10	97.92

Non-IID	LF attacks / %				RN attacks / %				RG attacks / %			
	0.4 Byzantine		0.6 Byzantine		0.4 Byzantine		0.6 Byzantine		0.4 Byzantine		0.6 Byzantine	
	Semi-Honest	Mali-cious	Semi-Honest	Mali-cious	Semi-Honest	Mali-cious	Semi-Honest	Mali-cious	Semi-Honest	Mali-cious	Semi-Honest	Mali-cious
FedAvg [21]	38.47	11.56	30.21	9.65	31.61	10.57	21.17	11.46	10.09	9.79	10.10	7.90
FLTrust [30]	67.26	9.58	47.49	9.43	44.40	10.22	27.70	8.78	11.35	12.00	11.35	9.22
Krum [10]	38.63	11.43	24.32	7.73	39.35	12.40	44.50	12.58	49.57	11.19	44.67	9.60
Trimmed [28]	49.17	12.91	41.89	14.93	48.32	14.47	35.88	12.56	38.66	10.10	11.35	10.32
Median [28]	27.96	9.74	15.68	19.74	49.31	9.74	42.33	9.74	35.32	9.74	30.20	9.74
FedInv [32]	66.22	13.62	52.71	13.87	35.35	19.70	18.39	12.92	24.32	14.24	24.32	13.14
CareFL [31]	39.61	11.61	27.48	12.00	41.18	11.47	32.69	15.60	60.90	11.86	63.86	12.69
zkFL (ours)	67.41	63.22	67.40	65.68	54.62	54.08	53.08	55.88	68.88	67.62	67.80	67.35

TABLE V
TIME CONSUMPTION

Time Consumption (s / epoch)								
Client Number	FedAvg [21]	Median [28]	Trimmed [28]	Krum [10]	FLTrust [30]	FedInv [32]	CareFL [31]	zkFL
10	142.09	146.28	149.35	149.49	152.76	158.59	154.57	190.29
20	255.60	283.84	295.59	319.09	313.16	349.32	364.76	404.53
50	739.87	759.49	793.05	798.23	812.88	812.29	830.73	976.56
100	1432.42	1498.47	1512.35	1529.74	1558.15	1606.89	1704.33	1911.93

TABLE VI
TIME BREAKDOWN OF ZERO-KNOWLEDGE PROOFS IN zkFL (S / EPOCH)

Time Consumption on MNIST Dataset (s / epoch)				
Client Number	Setup	Commit	Prove	Verify
10	4.64	5.98	0.18	4.32
20	4.75	12.64	0.24	8.50
50	6.01	32.06	0.41	22.64
100	6.35	64.15	0.92	45.92

Time Consumption on CIFAR Dataset (s / epoch)				
Client Number	Setup	Commit	Prove	Verify
10	4.80	112.15	0.88	4.32
20	4.92	225.60	1.75	8.64
50	5.96	564.10	4.38	21.60
100	6.21	1128.50	8.76	43.20

This advantage stems from the incorporation of zero-knowledge proofs, which rigorously validate the correctness of each aggregation step and ensure that only authenticated

model updates are integrated into the global model, thereby preserving robustness against adversarial manipulation and promoting stable convergence under heterogeneous and adversarial conditions.

C. Efficiency Evaluation

We evaluate the time efficiency of zkFL by analyzing the per-epoch training time and decomposing the ZKP components on both the MNIST and CIFAR datasets. As shown in Table VI and Fig. 4, zkFL introduces additional computational overhead compared with lightweight aggregation methods such as FedAvg and Krum, primarily due to the cryptographic operations in the Commit and Prove phases. However, this overhead is a reasonable trade-off, as zkFL provides stronger Byzantine-robustness and explicitly defends against malicious servers.

The detailed breakdown highlights different cost characteristics across stages. The Setup phase involves one-time initialization of public parameters and remains nearly constant across client numbers. The Commit phase dominates the total cost

because committing each gradient dimension requires exponentiation over elliptic curve groups, which scales with both the number of clients and the model dimensionality. The Prove phase adds only a small overhead since the zk-SNARK circuit encodes mainly linear relations, making proof generation efficient even for large client populations. Verification is relatively lightweight, as Groth16 proofs have constant size and can be checked with a small number of bilinear pairings, so the per-client verification cost grows linearly with N but remains moderate.

In the CIFAR experiments, Commit time is substantially higher than in MNIST due to the larger model size and more complex feature representation, which significantly increase the number of exponentiations required for commitments. For example, when $N = 100$, Commit accounts for over 95% of the total ZKP time in CIFAR, compared to 54% in MNIST. Despite this, the overall runtime overhead remains within a controllable range, ensuring that zkFL scales to large client populations while maintaining robust security guarantees.

VIII. CONCLUSION

In this paper, we propose zkFL, a Byzantine-robust FL framework tailored for deployment in low-altitude networks, where dynamic connectivity, heterogeneous data distribution, and adversarial threats are prevalent. Unlike prior defenses that assume server honesty, zkFL integrates zero-knowledge proofs into the aggregation process, enabling each client to independently verify the correctness of server-side aggregation. This verifiability ensures that even in the presence of a malicious server, global model updates remain trustworthy without relying on centralized trust assumptions.

Furthermore, zkFL accurately detects Byzantine clients in highly non-IID environments common to low-altitude networks, distinguishing adversarial behavior from natural deviations caused by data heterogeneity. By dynamically adjusting aggregation weights, it effectively suppresses malicious influence while preserving convergence.

Extensive experiments on multiple datasets demonstrate that zkFL consistently outperforms state-of-the-art Byzantine-robust methods, delivering both robust accuracy and verifiable security guarantees. These properties make zkFL particularly suitable for low-altitude network applications, where secure and reliable model training must be maintained despite unstable links, diverse data distributions, and adversarial participants.

REFERENCES

- [1] Y. Yang, Z. Zheng, K. Bian, L. Song, and Z. Han, "Real-time profiling of fine-grained air quality index distribution using UAV sensing," *IEEE Internet Things J.*, vol. 5, no. 1, pp. 186–198, Feb. 2018.
- [2] R. Doku, D. B. Rawat, and C. Liu, "Towards federated learning approach to determine data relevance in Big Data," in *Proc. IEEE 20th Int. Conf. Inf. Reuse Integration Data Sci.*, 2019, pp. 184–192.
- [3] R. Zhang et al., "Generative ai agents with large language model for satellite networks via a mixture of experts transmission," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 12, pp. 3581–3596, Dec. 2024.
- [4] F. Tang, Z. M. Fadlullah, N. Kato, F. Ono, and R. Miura, "AC-POCA: Anticoordination game based partially overlapping channels assignment in combined UAV and D2D-based networks," *IEEE Trans. Veh. Technol.*, vol. 67, no. 2, pp. 1672–1683, Feb. 2018.
- [5] T. Bai, J. Wang, Y. Ren, and L. Hanzo, "Energy-efficient computation offloading for secure UAV-edge-computing systems," *IEEE Trans. Veh. Technol.*, vol. 68, no. 6, pp. 6074–6087, Jun. 2019.
- [6] W. Yuan et al., "From ground to sky: Architectures, applications, and challenges shaping low-altitude wireless networks," 2025, *arXiv:2506.12308*.
- [7] G. Sun et al., "Online collaborative resource allocation and task offloading for multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 24, no. 11, pp. 11430–11448, Nov. 2025.
- [8] F. Song, Y. Gao, M. Zhao, C. Zhang, Z. Qin, and B. Xiao, "High-dimensional and secure spatial keyword query with arbitrary ranges in mobile cloud," *IEEE Trans. Mobile Comput.*, vol. 24, no. 11, pp. 12121–12136, Nov. 2025.
- [9] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 691–706.
- [10] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 118–128.
- [11] W. Huang, J. Liu, T. Li, S. Ji, D. Wang, and T. Huang, "FedCKE: Cross-domain knowledge graph embedding in federated learning," *IEEE Trans. Big Data*, vol. 9, no. 3, pp. 792–804, Jun. 2023.
- [12] J. Kang, Z. Xiong, D. Niyato, S. Xie, and J. Zhang, "Incentive mechanism for reliable federated learning: A joint optimization approach to combining reputation and contract theory," *IEEE Internet Things J.*, vol. 6, no. 6, pp. 10700–10714, Dec. 2019.
- [13] W. Zhang et al., "VSpatial: Enabling private and verifiable spatial keyword-based positioning in 6G-oriented IoT," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 10, pp. 2954–2969, Oct. 2024.
- [14] B. Li, Y. Wang, A. Singh, and Y. Vorobeychik, "Data poisoning attacks on factorization-based collaborative filtering," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2016, pp. 1893–1901.
- [15] V. Shejwalkar and A. Houmansadr, "Manipulating the byzantine: Optimizing model poisoning attacks and defenses for federated learning," in *Proc. Netw. Distrib. Syst. Secur.*, 2021.
- [16] X. Liu, T. J. Lim, and J. Huang, "Optimal Byzantine attacker identification based on game theory in network coding enabled wireless ad hoc networks," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 2570–2583, 2020.
- [17] G. Xu, H. Li, Y. Zhang, S. Xu, J. Ning, and R. H. Deng, "Privacy-preserving federated deep learning with irregular users," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 1364–1381, Mar./Apr. 2022.
- [18] M. Fang, X. Cao, J. Jia, and N. Gong, "Local model poisoning attacks to Byzantine-Robust federated learning," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 1605–1622.
- [19] M. Zhao, C. Zhang, T. Wu, J. Ni, X. Liu, and L. Zhu, "Revocable and privacy-preserving bilateral access control for cloud data sharing," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 5389–5404, 2024.
- [20] Y. Jiang, W. Zhang, and Y. Chen, "Data quality detection mechanism against label flipping attacks in federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 1625–1637, 2023.
- [21] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, 2017, pp. 1273–1282.
- [22] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Process. Mag.*, vol. 37, no. 3, pp. 50–60, May 2020.
- [23] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, "Data poisoning attacks against federated learning systems," in *Proc. Eur. Symp. Res. Comput. Secur.*, Springer, 2020, pp. 480–501.
- [24] L. Zhao, J. Jiang, B. Feng, Q. Wang, C. Shen, and Q. Li, "SEAR: Secure and efficient aggregation for byzantine-robust federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3329–3342, Sep./Oct. 2022.
- [25] H. Wang et al., "Attack of the tails: Yes, you really can backdoor federated learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 16070–16084.
- [26] L. Muñoz-González et al., "Towards poisoning of deep learning algorithms with back-gradient optimization," in *Proc. 10th ACM Workshop Artif. Intell. Secur.*, 2017, pp. 27–38.
- [27] O. Suciu, R. Marginean, Y. Kaya, H. Daume III, and T. Dumitras, "When does machine learning {FAIL}? Generalized transferability for evasion and poisoning attacks," in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 1299–1316.

- [28] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5650–5659.
- [29] R. Guerraoui et al., "The hidden vulnerability of distributed learning in byzantium," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 3521–3530.
- [30] X. Cao et al., "FLTrust: Byzantine-robust federated learning via trust bootstrapping," 2020, *arXiv:2012.13995*.
- [31] Q. Dong et al., "CareFL: Contribution guided byzantine-robust federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 9714–9729, 2024.
- [32] B. Zhao, P. Sun, T. Wang, and K. Jiang, "FedINV: Byzantine-robust federated learning by inverting local model updates," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 8, 2022, pp. 9171–9179.
- [33] C. Xie, S. Koyejo, and I. Gupta, "Zeno: Robust fully asynchronous SGD," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 10495–10 503.
- [34] C. Fung, C. J. Yoon, and I. Beschastnikh, "Mitigating sybils in federated learning poisoning," 2018, *arXiv:1808.04866*.
- [35] Z. Zhang, X. Cao, J. Jia, and N. Z. Gong, "FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 2545–2555.
- [36] Y. Lee, Y. Cho, W. Han, H. Bae, and Y. Paek, "FLGuard: Byzantine-robust federated learning via ensemble of contrastive models," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2023, pp. 65–84.
- [37] A. He, J. Wang, Z. Huang, and J. Xiao, "FedSmart: An auto updating federated learning optimization mechanism," in *Proc. Asia-Pacific Web Web-Age Inf. Manage. Joint Int. Conf. Web Big Data*. Springer, 2020, pp. 716–724.
- [38] R. Wang et al., "MudGuard: Taming malicious majorities in federated learning using privacy-preserving byzantine-robust clustering," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 53, no. 1, pp. 25–27, 2025.
- [39] T. D. Nguyen et al., "{FLAME}: Taming backdoors in federated learning," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 1415–1432.
- [40] A. R. Chowdhury, C. Guo, S. Jha, and L. van der Maaten, "EIFFEL: Ensuring integrity for federated learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2535–2549.
- [41] M. Fang, J. Liu, N. Z. Gong, and E. S. Bentley, "AFLGuard: Byzantine-robust asynchronous federated learning," in *Proc. 38th Annu. Comput. Secur. Appl. Conf.*, 2022, pp. 632–646.
- [42] M. Fang, Z. Liu, X. Zhao, and J. Liu, "Byzantine-robust federated learning over ring-all-reduce distributed computing," in *Proc. Companion Proc. ACM Web Conf.*, 2025, pp. 961–965.
- [43] P. Sun, X. Liu, Z. Wang, and B. Liu, "Byzantine-robust decentralized federated learning via dual-domain clustering and trust bootstrapping," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 24756–24765.
- [44] H. Wang, S. Wang, R. Wang, and P. Wang, "FEDCMP: Byzantine-robust federated learning through clustering model update parameters," *Inf. Sci.*, vol. 718, 2025, Art. no. 122377.
- [45] D. Alistarh, Z. Allen-Zhu, and J. Li, "Byzantine stochastic gradient descent," in *Proc. 32nd Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 4618–4628.
- [46] C. Xie et al., "Disrupting vision-language model-driven navigation services via adversarial object fusion," 2025, *arXiv:2505.23266*.
- [47] G. Baruch, M. Baruch, and Y. Goldberg, "A little is enough: Circumventing defenses for distributed learning," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 8635–8645.
- [48] F. Guo, F. R. Yu, H. Zhang, X. Li, H. Ji, and V. C. Leung, "Enabling massive IT toward 6G: A comprehensive survey," *IEEE Internet Things J.*, vol. 8, no. 15, pp. 11891–11915, Aug. 2021.
- [49] Z. Tong, J. Wang, X. Hou, C. Jiang, and J. Liu, "UAV-assisted covert federated learning over mmWave massive MIMO," *IEEE Trans. Wireless Commun.*, vol. 23, no. 9, pp. 11785–11798, Sep. 2024.
- [50] F. Zhao, B. Yang, C. Li, C. Zhang, L. Zhu, and G. Liang, "Two-layer consensus based on master-slave consortium chain data sharing for internet of vehicles," 2024, *arXiv:2411.10680*.
- [51] J. Li, G. Sun, L. Duan, and Q. Wu, "Multi-objective optimization for UAV swarm-assisted IoT with virtual antenna arrays," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 4890–4907, May 2024.
- [52] Z. Wei et al., "UAV-assisted data collection for Internet of Things: A survey," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 15460–15483, Sep. 2022.
- [53] M. Barni, K. Kallas, and B. Tondi, "A new backdoor attack in CNNs by training set corruption without label poisoning," in *Proc. IEEE Int. Conf. Image Process.*, 2019, pp. 101–105.
- [54] A. Sharma and N. Marchang, "Probabilistic sign flipping attack in federated learning," in *Proc. 15th Int. Conf. Comput. Commun. Netw. Technol.*, 2024, pp. 1–6.
- [55] A. Yazdinejad, A. Dehghantanha, H. Karimipour, G. Srivastava, and R. M. Parizi, "A robust privacy-preserving federated learning model against model poisoning attacks," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 6693–6708, 2024.
- [56] Z. Li, L. Liu, J. Zhang, and J. Liu, "Byzantine-robust federated learning through spatial-temporal analysis of local model updates," in *Proc. IEEE 27th Int. Conf. Parallel Distrib. Syst.*, 2021, pp. 372–379.
- [57] C. Xie et al., "DBA: Distributed backdoor attacks against federated learning," in *Proc. 8th Int. Conf. Learn. Representations*, 2020.
- [58] C. Fung, C. J. Yoon, and I. Beschastnikh, "The limitations of federated learning in sybil settings," in *Proc. 23rd Int. Symp. Res. Attacks, Intrusions Defenses*, 2020, pp. 301–316.
- [59] J. Wang et al., "Generative AI based secure wireless sensing for ISAC networks," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 5195–5210, 2025.
- [60] G. Sun et al., "Aerial reliable collaborative communications for terrestrial mobile users via evolutionary multi-objective deep reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 24, no. 7, pp. 5731–5748, Jul. 2025.
- [61] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," in *Proc. 29th Int. Conf. Mach. Learn.*, 2012, pp. 1807–1814.
- [62] A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, "Analyzing federated learning through an adversarial lens," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 634–643.
- [63] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2020, pp. 2938–2948.
- [64] Z. Sun, P. Kairouz, A. T. Suresh, and H. B. McMahan, "Can you really backdoor federated learning?," 2019, *arXiv:1911.07963*.
- [65] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [66] Y. Chen, L. Su, and J. Xu, "Distributed statistical machine learning in adversarial settings: Byzantine gradient descent," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 1, no. 2, pp. 1–25, 2017.
- [67] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "Scaffold: Stochastic controlled averaging for federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 5132–5143.
- [68] B. Al-Bataineh, S. Schindler, D. Peters, M. Paradies, and M. Pohl, "A systematic review of federated statistical heterogeneity in UAV applications," *CEAS Aeronautical J.*, pp. 1–23, 2025.
- [69] P. Li, Z. Liu, L. Chang, J. Peng, and Y. Wu, "FedBA: Non-IID federated learning framework in UAV networks," in *Proc. Secure Edge Fog Comput. Enabled AI IoT Smart Cities: Includes Sel. Papers Int. Conf. Adv. Comput. Next-Gener. Commun.*, 2023, pp. 121–131.
- [70] W. Y. B. Lim et al., "Federated learning in mobile edge networks: A comprehensive survey," *IEEE Commun. Surveys Tut.*, vol. 22, no. 3, pp. 2031–2063, thirdquarter 2020.
- [71] G. Sun et al., "Joint task offloading and resource allocation in aerial-terrestrial UAV networks with edge and fog computing for post-disaster rescue," *IEEE Trans. Mobile Comput.*, vol. 23, no. 9, pp. 8582–8600, Sep. 2024.
- [72] D. Rothchild et al., "FetchSGD: Communication-efficient federated learning with sketching," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 8253–8265.
- [73] D. Pasquini, D. Francati, and G. Ateniese, "Eluding secure aggregation in federated learning via model inconsistency," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2429–2443.
- [74] Z. Pan et al., "Feature-based machine unlearning for vertical federated learning in IoT networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 6, pp. 5031–5044, Jun. 2025.
- [75] E. M. El-Mhamdi, S. Farhadkhani, R. Guerraoui, A. Guirguis, L.-N. Hoang, and S. Rouault, "Collaborative learning in the jungle (decentralized, byzantine, heterogeneous, asynchronous and nonconvex learning)," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 25044–25057.

- [76] D. Cajaraville-Aboy, A. Fernández-Vilas, R. P. Dráz-Redondo, and M. Fernández-Veiga, “Byzantine-robust aggregation for securing decentralized federated learning,” 2024, *arXiv:2409.17754*.
- [77] L. Yuan, Z. Wang, L. Sun, P. S. Yu, and C. G. Brinton, “Decentralized federated learning: A survey and perspective,” *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34617–34638, Nov. 2024.
- [78] S. Wang, D. Li, J. Geng, Y. Gu, and Y. Cheng, “Impact of network topology on the performance of DML: Theoretical analysis and practical factors,” in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 1729–1737.
- [79] A. Nedić, A. Olshevsky, and M. G. Rabbat, “Network topology and communication-computation tradeoffs in decentralized optimization,” *Proc. IEEE*, vol. 106, no. 5, pp. 953–976, May 2018.
- [80] M. Salehi and E. Hossain, “Federated learning in unreliable and resource-constrained cellular wireless networks,” *IEEE Trans. Commun.*, vol. 69, no. 8, pp. 5136–5151, Aug. 2021.
- [81] S. Ganguli, Z. Zhou, C. G. Brinton, and D. I. Inouye, “Fault tolerant serverless VFL over dynamic device environment,” 2023, *arXiv:2312.16638*.
- [82] S. Guo et al., “Byzantine-resilient decentralized stochastic gradient descent,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 6, pp. 4096–4106, Jun. 2022.
- [83] L. He, S. P. Karimireddy, and M. Jaggi, “Byzantine-robust decentralized learning via CLIPPedGossip,” 2022, *arXiv:2202.01545*.
- [84] O. Marfoq, C. Xu, G. Neglia, and R. Vidal, “Throughput-optimal topology design for cross-silo federated learning,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 19478–19487.
- [85] A. Koloskova, N. Loizou, S. Boreiri, M. Jaggi, and S. Stich, “A unified theory of decentralized sgd with changing topology and local updates,” in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 5381–5393.
- [86] M. Assran, N. Loizou, N. Ballas, and M. Rabbat, “Stochastic gradient push for distributed deep learning,” in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 344–353.
- [87] Y. Huang et al., “Personalized cross-silo federated learning on non-IID data,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 9, 2021, pp. 7865–7873.
- [88] Y. Miao, Z. Liu, H. Li, K.-K. R. Choo, and R. H. Deng, “Privacy-preserving byzantine-robust federated learning via blockchain systems,” *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 2848–2861, 2022.
- [89] S. P. Karimireddy et al., “Breaking the centralized barrier for cross-device federated learning,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 28663–28676.
- [90] M. H. ur Rehman, A. M. Dirir, K. Salah, E. Damiani, and D. Svetinovic, “Trustfed: A framework for fair and trustworthy cross-device federated learning in IIoT,” *IEEE Trans. Ind. Informat.*, vol. 17, no. 12, pp. 8485–8494, Dec. 2021.
- [91] P. Rogaway and T. Shrimpton, “Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance,” in *Proc. Int. Workshop Fast Softw. Encryption*. Springer, 2004, pp. 371–388.
- [92] A. J. Menezes, P. C. Van Oorschot, and S. A. Vanstone, *Handbook of Appl. Cryptography*. Boca Raton, FL, USA: CRC press, 2018.
- [93] I. Damgård and M. Jurik, “A generalisation, a simplification and some applications of Paillier’s probabilistic public-key system,” in *Proc. Int. Workshop Public Key Cryptogr.*, 2001, pp. 119–136.
- [94] T. P. Pedersen, “Non-interactive and information-theoretic secure verifiable secret sharing,” in *Proc. Annu. Int. Cryptol. Conf.*, 1991, pp. 129–140.
- [95] D. F. Aranha, E. M. Bennedsen, M. Campanelli, C. Ganesh, C. Orlandi, and A. Takahashi, “ECLIPSE: Enhanced compiling method for Pedersen-committed zkSNARK engines,” in *Proc. IACR Int. Conf. Public-Key Cryptogr.*, 2022, pp. 584–614.
- [96] Z. Wang, N. Dong, J. Sun, W. Knottenbelt, and Y. Guo, “ZKFL: Zero-knowledge proof-based gradient aggregation for federated learning,” *IEEE Trans. Big Data*, vol. 11, no. 2, pp. 447–460, Apr. 2025.
- [97] X. Sun, F. R. Yu, P. Zhang, Z. Sun, W. Xie, and X. Peng, “A survey on zero-knowledge proof in blockchain,” *IEEE Netw.*, vol. 35, no. 4, pp. 198–205, Jul/Aug. 2021.
- [98] J. Kilian, “A note on efficient zero-knowledge proofs and arguments,” in *Proc. 24th Annu. ACM Symp. Theory Comput.*, 1992, pp. 723–732.
- [99] O. Goldreich and Y. Oren, “Definitions and properties of zero-knowledge proof systems,” *J. Cryptol.*, vol. 7, no. 1, pp. 1–32, 1994.
- [100] C. Weng, K. Yang, J. Katz, and X. Wang, “Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits,” in *Proc. IEEE Symp. Secur. Privacy*, 2021, pp. 1074–1091.
- [101] E. B. Sasson et al., “ZeroCash: Decentralized anonymous payments from bitcoin,” in *Proc. IEEE Symp. Secur. Privacy*, 2014, pp. 459–474.
- [102] I. Damgård, V. Pastro, N. Smart, and S. Zakarias, “Multiparty computation from somewhat homomorphic encryption,” in *Proc. Annu. Cryptol. Conf.*, 2012, pp. 643–662.
- [103] M. Mozaffari, W. Saad, M. Bennis, Y.-H. Nam, and M. Debbah, “A tutorial on UAVs for wireless networks: Applications, challenges, and open problems,” *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2334–2360, thirdquarter 2019.
- [104] Y. Zeng, Q. Wu, and R. Zhang, “Accessing from the sky: A tutorial on UAV communications for 5G and beyond,” *Proc. IEEE*, vol. 107, no. 12, pp. 2327–2375, Dec. 2019.
- [105] Y. LeCun and C. Cortes, “MNIST handwritten digit database,” 2010. [Online]. Available: <http://yann.lecun.com/exdb/mnist/>
- [106] A. Krizhevsky et al., “Learning Multiple Layers of Features From Tiny Images,” 2009.



Xiangyun Tang (Member, IEEE) received the B.Eng. degree in computer science from Minzu University of China, Beijing, China in 2016, and the Ph.D. degree in cyberspace security from the Beijing Institute of Technology, Beijing, in 2022. She is currently an Associate Professor with the School of Information Engineering, Minzu University of China. She has authored or coauthored more than 30 papers including AAAI, IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE NETWORK, IEEE WIRELESS COMMUNICATIONS LETTERS, IEEE GLOBECOM, IEEE ICC, and IEEE WCNC. Her research interests include secure multi-party computation and artificial intelligence security. She was the recipient of the Best Paper Award from IEEE ICBCTIS 2023, ICA3PP 2024, and Outstanding Paper Award from IEEE iThings 2023. She was the Guest Editor for multiple Journals, TPC Chair, and PC Member for international conferences and workshops.



Minyang Li received the B.Eng. degree in network engineering from Southwest Minzu University, Chengdu, China in 2023. He is currently working toward the Graduate degree with the School of Information Engineering, Minzu University of China, Beijing, China. His research interests include federated learning and privacy protection. He has authored or coauthored the paper *Exploring the Impact of Non-IID on Federated Learning* at the ICBCTIS 2023 conference. He was the recipient of the Best Paper Award.



Tao Zhang received the B.S. degree in Internet of Things engineering from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, and from the Queen Mary University of London, London, U.K., in 2018, and the Ph.D. degree in computer science and technology from BUPT, in 2023. He is currently an Associate Professor with the School of Cyberspace Science and Technology, Beijing Jiaotong University. His publications include ESI highly cited paper and well-archived international journals and proceedings, such as IEEE COMMUNICATIONS SURVEYS AND TUTORIALS, IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING and IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS. His research interests include network security, moving target defense, and federated learning. He was the Guest Editor of *Electronics* and *Chinese Journal of Network and Information Security*, and the TPC Chair and a PC Member for some international conferences and workshops. He was the recipient of the Best Paper Award from NaNA 2018, IWCNC 2021, DIONE 2024, and ICA3PP 2024, and Outstanding Paper Award from iThings 2023, and SmartCity 2024, his Ph.D. thesis was the recipient of the Outstanding Doctoral Dissertation by BUPT in 2023.



Yijing Lin (Member, IEEE) received the bachelor's degree from North China Electric Power University, Beijing, China. She is currently working toward the Ph.D. degree with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing.



Chuan Zhou received the Ph.D. degrees in Tianjin University, Tianjin, China, in 2024. He is currently a Professor with the Department of Minzu University of China, Beijing, China. His research interests include artificial intelligence and Big Data, intelligent control, and engineering integrated application of wheeled mobile robots.



Liehuang Zhu (Senior Member, IEEE) received the Ph.D. degree in computer science from the Beijing Institute of Technology, Beijing, China, in 2004. He is currently a Professor and the Dean with the School of Cyberspace Science and Technology, Beijing Institute of Technology. He has authored or coauthored more than 150 peer-reviewed journal or conference papers. His research interests include security protocol analysis and design, wireless sensor networks, and cloud computing. Dr. Zhu was the recipient of the number of IEEE Best Paper awards, including

IWQoS 17' and TrustCom 18'. He is also the Area Editor, Associate Editor, or Academic Editor of multiple SCI-index journals, such as IEEE COMMUNICATIONS SURVEYS & TUTORIALS and IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY.



Xiangzhi Liu received the Ph.D. degree and is currently a Research Professor and Ph.D./M.S. degree Supervisor with the Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Qilu University of Technology (Shandong Academy of Sciences), Jinan, China. He is also a Taishan Industry Leading Talent and Jinan Haiyou Distinguished Industry Leader. His research focuses on AI security defense, blockchain-based data management, and industrial Internet data governance.